Computational Statistics and Machine Learning Coursework 2025-26
MSc in Statistics, Imperial College London

Ludovico Amedeo Panariello

06060027

Contents

Question 1 (50% of total marks)

Consider the following target density:

$$f(x) = k\Big(\exp[-3(x-2)] + \exp\big[-(x-30)^2\big] + \exp\big[-(x-20)^2/0.01\big] \Big)\, \mathbb{1}_{\{x \geq 2\}}$$

where k is a normalising constant and $\mathbb{1}_{\{\cdot\}}$ denotes the indicator function.

(a) Implement a random-walk Metropolis–Hastings algorithm with normal noise to sample from this target distribution. Investigate the impact of the variance of the random noise on the convergence and mixing quality of the chain. Discuss your findings.

(b) Devise and implement a Metropolis–Hastings algorithm which does not propose values of x outside of the support of f. Precisely describe this algorithm with mathematical equations. Verify graphically that the MCMC algorithm has reached convergence.

(c) Devise and implement a parallel tempering algorithm to sample from f. Provide a pseudo-code describing the algorithm you constructed. Precisely state the temperatures you are using as well as the transition kernels for each chain. Verify graphically that the sampler is exploring the full space and correctly sampling from the target distribution.

(d) Compare the three algorithms (from part (a), (b) and (c)) in terms of convergence, mixing properties of the resulting Markov chains and computational costs.

(e) Suppose that I want to estimate $E(X)$ for X following a distribution with p.d.f. f. Denote by $\hat{I}_n^{(a)}$, $\hat{I}_n^{(b)}$ and $\hat{I}_n^{(c)}$ the estimate of $E(X)$ using a chain of length n produced by the MCMC algorithms from questions (a), (b) and (c) respectively. Write down the equation of a 95% confidence interval for $\hat{I}_n^{(a)}$, $\hat{I}_n^{(b)}$ and $\hat{I}_n^{(c)}$.

Produce three plots featuring $\hat{I}_n^{(a)}$, $\hat{I}_n^{(b)}$ and $\hat{I}_n^{(c)}$ respectively as a function of n along with a 95% confidence interval for these estimates. Assess how the three estimates differ from each other, and compare them to the reference value of $E[X]$ obtained by numerical integration.

Question 2 (50% of total marks)

In this question I explore quantile regression, using the following asymmetric loss

$$\ell_\tau(y, q) = \begin{cases} \tau(y-q), & y > q, \\ (\tau-1)(y-q), & y \leq q, \end{cases}$$

for a quantile level $\tau \in (0,1)$. Let

$$R_\tau(q; x) = \mathbb{E}\big[\, \ell_\tau(Y, q) \mid X = x \,\big]$$

denote the conditional risk. It is known that

$$q_\tau^*(x) := \arg\min_q R_\tau(q; x)$$

is the conditional τ-quantile of $Y \mid X = x$. I therefore use empirical risk minimisation with this loss to learn an estimator $f_\tau(x)$ for the conditional τ-quantile.

I use the `Boston` dataset from the `MASS` package in R. Let the response be `medv` (median house price). Our goal is to predict the conditional quantiles of `medv` based on the features `lstat` and `rm`. Randomly split the data into a training set (70%) and a test set (30%). Throughout this question, you must implement the loss yourself. You may use generic optimisation routines such as `optim`, but you may not use pre-built quantile regression or distributional regression packages. In each part, explain how you performed the optimisation (the choice of optimiser, the initial state, any associated settings, and how convergence was assessed).

(a) Consider the baseline linear model

$$f_\tau^{(0)}(x) = \beta_{0,\tau} + \beta_{1,\tau}\, \texttt{lstat} + \beta_{2,\tau}\, \texttt{rm}.$$

- Implement the loss and its empirical mean in R. Fit the parameters $(\beta_{0,\tau}, \beta_{1,\tau}, \beta_{2,\tau})$ for each $\tau \in \{0.1, 0.5, 0.9\}$ by minimising the empirical loss over the training set using `optim` and report the estimated coefficients.
- For each τ evaluate the empirical loss on the test set.

- The empirical coverage
$$\widehat{C}_\tau^{(0)} = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} \mathbb{1}\{Y_i \le f_\tau^{(0)}(X_i)\},$$
measures the proportion of test responses whose actual value lies below the predicted τ-quantile. Since a true τ-quantile satisfies
$$\Pr\left[\, Y \le q_\tau(X) \,\right] = \tau,$$
I ideally expect $\widehat{C}_\tau^{(0)} \approx \tau$ for a well-calibrated model. For each τ, compute the empirical coverage and compare it with the target level τ.

(b) Extend the model by introducing additional quadratic terms $\texttt{lstat}^2$, $\texttt{rm}^2$ and the interaction term $\texttt{lstat}\times\texttt{rm}$. Let $f_\tau^{(1)}$ be the resulting linear model.

- Fit $f_\tau^{(1)}$ for each $\tau \in \{0.1, 0.5, 0.9\}$ by minimising the empirical loss over the training set and report the coefficients.
- Evaluate the test loss and empirical coverage.

(c) Introduce an ℓ_2 (ridge) penalty on the coefficients of the nonlinear model. For a given $\lambda \ge 0$, define the ridge-penalised empirical risk
$$R_{\tau,\lambda}^{\text{ridge}}(\beta) = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} \ell_\tau(Y_i, f_\tau^{(1)}(X_i;\beta)) + \lambda\|\beta\|_2^2.$$

- For each $\tau \in \{0.1, 0.5, 0.9\}$ and a set of penalty values λ within $[10^{-4}, 10^{-1}]$, fit the ridge-penalised model over the training set, using k-fold cross-validation on the empirical risk within the training set to select λ for each τ.
- For the selected λ, report the ridge-penalised coefficients $\hat{\beta}_{\tau,\lambda}^{\text{ridge}}$ and the test loss and empirical coverage.

(d) Discuss and interpret the relative performances of each model, in terms of (i) coefficient magnitudes, (ii) test loss, (iii) overfitting and (iv) empirical coverage. In particular, explain the reason for the outcomes you have observed.

(e) The true quantiles never cross, i.e.
$$q_\tau(x) < q_{\tau'}(x) \quad \text{for all } x \text{ and } 0 \le \tau < \tau' \le 1.$$

- Will the quantile prediction models you have trained automatically satisfy a similar relationship in general? Explain your answer.
- Suggest an approach for training a quantile prediction model for quantiles $\tau \in \{0.1, 0.5, 0.9\}$ which discourages such crossings. Provide details of the associated optimisation problem, the loss and any other relevant details. (Note that you do not need to implement this, just describe it.)

1.1 Question 1

The target density in the question can be written as

$$f(x) = k\left\{\exp[-3(x-2)] + \exp[-(x-30)^2] + \exp\left[-(x-20)^2/0.01\right]\right\}\mathbf{1}_{\{x\geq 2\}},$$

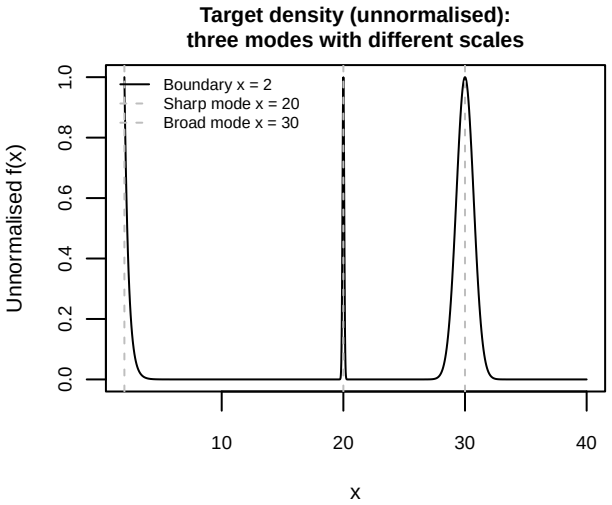
where k is an unknown normalising constant and $\mathbf{1}_{\{x\geq 2\}}$ enforces the support constraint $x \geq 2$. In a Metropolis–Hastings algorithm I never need the value of k, because the acceptance probability only involves ratios of the target,

$$\alpha(x,y) = \min\left\{1,\ \frac{f(y)}{f(x)}\right\} = \min\left\{1,\ \frac{\tilde{f}(y)}{\tilde{f}(x)}\right\},$$

so the common factor k cancels. I therefore work with the unnormalised density $\tilde{f}(x)$, defining it directly in R as `target_unnorm(x)` and setting it to zero for $x < 2$ to encode the support.

For later use in the Metropolis–Hastings ratio I also define `log_target_unnorm(x)`, the log of the unnormalised target as evaluating log–densities and subtracting them is numerically more stable than working with very small probabilities on the original scale.

To get an initial qualitative understanding of how challenging this target is for MCMC, I first plot the unnormalised density $\tilde{f}(x)$ over a grid on $[2, 40]$.



Target structure.
The plot on the left shows three distinct regions of high probability:

- an exponential tail for $x \in [2, 8]$ with non–negligible probability mass;
- a sharp peak at $x \approx 20$ with very narrow width ($\sigma = 0.1$);
- a broad peak at $x \approx 30$ with moderate width ($\sigma = 1$).

To assess whether each MCMC algorithm correctly samples from the target, I compute the theoretical probability mass in each region by analytically integrating the three component densities:

$$I_{\text{tail}} = \int_2^\infty \exp[-3(x-2)]\,dx = \tfrac{1}{3}, \quad I_{\text{broad}} = \int_{-\infty}^\infty \exp[-(x-30)^2]\,dx = \sqrt{\pi}, \quad I_{\text{narrow}} = \int_{-\infty}^\infty \exp[-(x-20)^2/0.01]\,dx = 0.1\sqrt{\pi}.$$

Dividing each by the total $I_{\text{total}} = I_{\text{tail}} + I_{\text{broad}} + I_{\text{narrow}}$ gives theoretical proportions of approximately 14.6% (tail), 77.6% (broad mode at $x \approx 30$), and 7.8% (narrow mode at $x \approx 20$). These reference values allow me to verify that a well-mixing chain visits each region with the correct frequency.

This heterogeneous structure is challenging for sampling because it requires the algorithm to explore both the low–probability tail region and the high–probability peaks, bridge gaps between distant modes and handle simultaneously a very narrow mode and a broader one, which puts competing requirements on the proposal step size.

1.2 Question 1a – Random–walk Metropolis–Hastings on the given target

1.2.1 Algorithm specification and numerical considerations

I employ a standard random–walk Metropolis–Hastings scheme with symmetric Gaussian proposals

$$Y \mid X_{t-1} \sim N(X_{t-1}, \sigma^2).$$

Because the proposal kernel is symmetric, the Hastings ratio simplifies: the proposal densities cancel and the MH acceptance probability becomes

$$\alpha(x, y) = \min\left\{1,\ \frac{\tilde{f}(y)}{\tilde{f}(x)}\right\}.$$

A direct implementation of the ratio $\tilde{f}(y)/\tilde{f}(x)$ is numerically dangerous. In regions of low probability density $\tilde{f}(\cdot)$ can be extremely small, and computing ratios of tiny numbers leads to underflow or overflow. To avoid this, I work entirely on the logarithmic scale.

I define the log–target $\log \tilde{f}(x)$ via a helper function that carefully handles the three exponential terms and respects the support constraint $x \geq 2$. The acceptance is then based on

$$\log \alpha(x, y) = \log \tilde{f}(y) - \log \tilde{f}(x).$$

If this log–ratio exceeds 0, I accept with probability 1 (the new state is better). Otherwise I accept with probability $\exp(\log \alpha)$, which is a number in $(0, 1)$. This approach eliminates underflow because:

- I compute logarithms of potentially small densities (which are negative numbers, hence harmless).
- I exponentiate only a single scalar log–ratio (already in the range $(-\infty, 0]$), so numerical precision is maintained.

This function returns the full trajectory and the empirical acceptance rate, both essential for diagnosing algorithm performance.

1.2.2 Empirical behaviour for different proposal scales: exploration of the tuning landscape

The proposal step size σ is the critical free parameter in random–walk MH. Intuitively:

- Very small σ gives highly local moves with short correlation times in the spatial sense (each step is tiny) but long correlation times in the temporal sense (the chain needs many steps to explore); the acceptance rate is high.
- Very large σ gives aggressive, far-ranging proposals that decorrelate spatially but are rejected frequently because they often land in low-probability regions; the acceptance rate is low.

Between these extremes lies an optimal regime where the chain balances proposal size against acceptance probability to maximise the rate of independent information extraction.

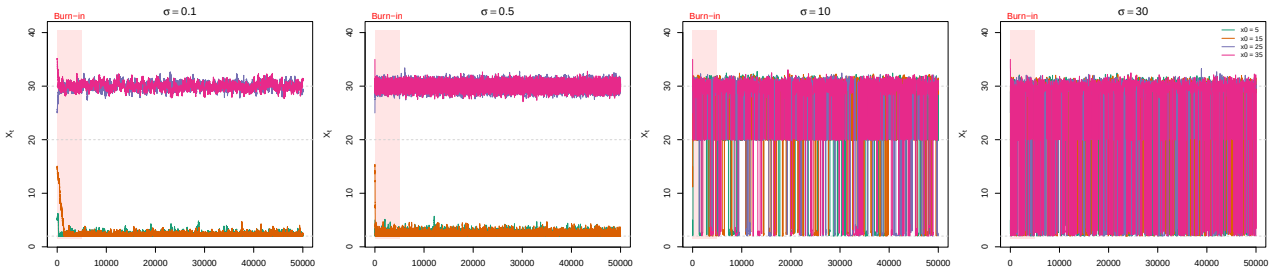
To explore this tuning landscape I run the RWMH algorithm[1] with four proposal standard deviations $\sigma \in \{0.1, 0.5, 10, 30\}$. For each σ I simulate 4 independent chains with distinct starting values $x_0 \in \{5, 15, 25, 35\}$ to test whether the algorithm converges from diverse initial conditions (a chain starting in the left tail should eventually explore the peaks near 20 and 30, and vice versa). Each chain has length $n = 50{,}000$ iterations. After a burn–in period of 5,000 iterations (during which the chain transients toward stationarity), I analyse mixing using multiple complementary diagnostics.

```
#> ACCEPTANCE RATE ANALYSIS
#> sigma =  0.1: mean acc. = 87.44%  (per chain: 79.24%, 79.78%, 95.27%, 95.47%)
#> sigma =  0.5: mean acc. = 59.93%  (per chain: 41.09%, 41.33%, 78.30%, 79.01%)
#> sigma = 10.0: mean acc. = 8.36%  (per chain: 8.36%, 8.50%, 8.40%, 8.18%)
#> sigma = 30.0: mean acc. = 3.58%  (per chain: 3.60%, 3.58%, 3.59%, 3.56%)
```

As expected from theory, the mean acceptance rate decreases monotonically as σ increases: proposals of $\sigma = 0.1$ or 0.5 (very local moves) are accepted nearly always, while $\sigma = 10$ or 30 (aggressive jumps) face much higher rejection rates around 8%–4%. However, this raw acceptance-rate ranking does not tell us which σ is best for sampling. A high acceptance rate can reflect high stickiness (chains moving locally) rather than efficient exploration. Conversely, a low acceptance rate paired with large spatial moves might yield more independent samples. To evaluate efficiency fairly, I must examine mixing structure.

1.2.3 Trace plots and qualitative mixing assessment

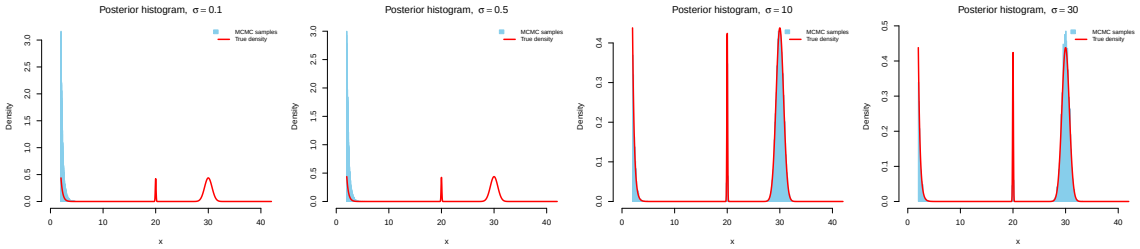
A trace plot visualises the trajectory $X_1, X_2, \ldots, X_n$ of the Markov chain over iteration number. By overlaying four chains with different starting values (coloured differently), I can visually inspect convergence and spatial exploration patterns. The reference lines at $x \in \{2, 20, 30\}$ mark the boundary and the modes.



[1] Throughout this coursework, I use `set.seed(123)` for all MCMC computations to ensure reproducibility.

1.2.4 Histogram comparison with true density

Beyond trace plots, I assess sampling quality by comparing the empirical histogram of MCMC samples against the normalised target density (normalised via numerical integration of $\tilde{f}$ over $[2, 50]$). For the well-mixing chains ($\sigma = 10, 30$), I [2] since they all explore the same stationary distribution. For the poorly-mixing chains ($\sigma = 0.1, 0.5$), pooling would artificially combine samples from chains stuck in different local modes, so I instead show the histogram from a single representative chain (chain 1, starting at $x_0 = 5$).



1.2.5 Quantitative mode coverage

The histograms provide a visual diagnostic; I now quantify sampling accuracy numerically by comparing the empirical proportion of samples in each region against the theoretical proportions derived from the analytical integration (14.6% tail, 7.8% narrow mode, 77.6% broad mode). For the well-mixing samplers ($\sigma = 10, 30$), I pool all four chains. For the poorly-mixing samplers ($\sigma = 0.1, 0.5$), I report proportions from a single chain (chain 1, starting at $x_0 = 5$) to illustrate how a chain stuck in one mode fails to recover the correct distribution.

Table 1: Mode coverage for RWMH samplers. For $\sigma = 0.1, 0.5$ (poorly-mixing), proportions are from chain 1 only; for $\sigma = 10, 30$ (well-mixing), all four chains are pooled.

Region	Theoretical	$\sigma = 0.1$	$\sigma = 0.5$	$\sigma = 10$	$\sigma = 30$
Tail ($x < 12$)	14.6%	100.0%	100.0%	15.3%	13.6%
Mode $x \approx 20$ ($18 \leq x \leq 22$)	7.8%	0.0%	0.0%	8.0%	7.3%
Mode $x \approx 30$ ($x > 25$)	77.6%	0.0%	0.0%	76.8%	79.1%

For $\sigma = 0.1$ and $\sigma = 0.5$, the table confirms the mixing failure: chain 1 (starting at $x_0 = 5$) places nearly all its mass in the tail region, completely missing the modes at $x \approx 20$ and $x \approx 30$. This starkly contrasts with the well-mixing samplers, which recover proportions close to the theoretical values.

Interpretation of trace patterns by proposal scale:

$\sigma = 0.1$: All four chains move in extremely small steps and remain essentially confined to their starting neighbourhoods for the full 50,000 iterations. The chain starting at $x_0 = 5$ oscillates tightly in the left tail region near the boundary, exploring the exponential component around $x \approx 2$. The chain starting at $x_0 = 15$ is gradually "pulled" towards the same broad exponential region on the left, while the chain starting at $x_0 = 25$ drifts towards the wide Gaussian mode around $x \approx 30$; in both cases the chains effectively bypass the very sharp peak at $x \approx 20$, whose thin tails make it much harder to hit with tiny random–walk moves. The chain starting at $x_0 = 35$ likewise settles near the third mode with minimal excursions. The y-axis range shows four distinct "bands" representing four non-communicating chains. This represents failure of mixing: despite a near-perfect acceptance rate ($ _acceptance~87%), the chains are completely sticky to the broader mode closest to their starting value. The algorithm proposes minuscule steps that are almost always accepted, but after 50,000 such tiny moves the chain has barely moved in state space and none of the chains explore the other modes.

$\sigma = 0.5$: The chains still remain largely localised, but there is noticeably more vertical range for the modes that exhibit some variance (third mode). However, large transitions between distant modes are still non-existent. With acceptance around 59%, proposals are still small enough that the chain is sticky despite slightly better local exploration than $\sigma = 0.1$.

$\sigma = 10$: A dramatic change. All four chains now regularly and visibly hop between all three regions (tail, broad peak near 30, and sharp peak near 20). The acceptance rate has dropped to around 8%, yet the traces show frequent large jumps across the state space. Each accepted proposal now represents a substantial move; rejected proposals create the flat segments slightly visible in the trace. All four chains, irrespective of starting value, converge toward the same regions of support, demonstrating convergence in distribution.

$\sigma = 30$: The mean acceptance rate is lowest at around 3.58%, yet the chains mix similarly to $\sigma = 10$. I must examine the ACF decay and convergence diagnostics ($\hat{R}$ and ESS) to quantify any advantage of more aggressive proposals against the computational cost of wasting more proposals.

Convergence vs. stickiness: The contrast is striking. High acceptance ($\sigma = 0.1, 0.5$) paired with local proposals creates sticky chains that converge (in the sense of high acceptance) to local distributions around their starting values. Lower acceptance ($\sigma = 10, 30$) with aggressive proposals enables chains to communicate between distant modes and extract information about the full target, despite most proposals being rejected.

[2] Pooling across chains is performed here to visualize the mcmc samples in one single histogram for each sigma

1.2.6 Autocorrelation functions: quantifying temporal dependence

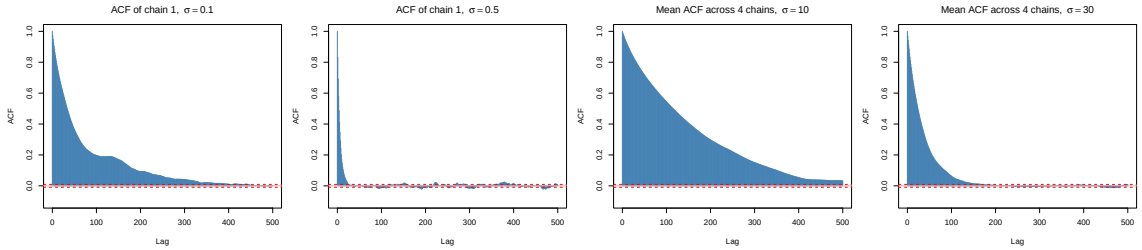
While trace plots show mixing behaviour visually, the autocorrelation function (ACF) provides a quantitative measure. For a stationary chain (X_t), the ACF at lag k is defined as

$$\rho_k = \mathrm{Corr}(X_t, X_{t+k}) = \frac{\mathrm{Cov}(X_t, X_{t+k})}{\mathrm{Var}(X_t)}.$$

By stationarity, ρ_k depends only on k, not on t. For an ideal i.i.d. sample, $\rho_k = 0$ for all $k > 0$. For a Markov chain, ρ_k typically decays toward zero as $k \to \infty$, with the rate of decay reflecting mixing efficiency: slow decay indicates strong persistence (many iterations needed to decorrelate); rapid decay indicates efficient mixing.

Following the same logic as before with histograms, I adopt different ACF strategies depending on mixing quality:

- **For** $\sigma = 0.1, 0.5$ (poorly mixing chains trapped in different modes): I show the ACF of a **single representative chain** (chain 1). Averaging ACFs across chains that sample from different local distributions would be meaningless.
- **For** $\sigma = 10, 30$ (well-mixing chains): I compute the ACF separately for each of the four chains and then **average** across chains.[3] This reduces noise while preserving the true within-chain temporal dependence. For poorly-mixing chains, I intentionally avoid any pooling or averaging to expose the failure mode.



Interpretation of ACF decay patterns:

Sigma = 0.1: The ACF remains essentially constant at approximately 1.0 across all lags up to 500. This extreme persistence means that samples separated by hundreds of iterations are almost as correlated as consecutive samples. The chain moves in such tiny steps that it "remembers" its starting configuration for the entire run. Note that this ACF is from a single chain trapped in one mode; the other chains (trapped in different modes) would show similar patterns of extreme autocorrelation.

Sigma = 0.5: Similar to $\sigma = 0.1$, the ACF remains very close to 1 and decays extremely slowly. Even at lag 500, the correlation is still substantial. This confirms that individual chains, while internally consistent, are highly autocorrelated and provide very few independent samples.

Sigma = 10: A dramatic contrast. The mean ACF drops from 1 at lag 0 to approximately 0.5 within 150 lags, continues declining toward near-zero by lag 500. This rapid decay reflects the global mode-hopping visible in the trace plots: large accepted jumps cause the chains to "forget" their past quickly. Averaging across chains is meaningful here because all four chains explore the same distribution.

Sigma = 30: The fastest ACF decay. The correlation drops to near-zero within roughly 100 lags. Beyond lag 200, the mean ACF oscillates around zero, indicating that samples separated by more than ~100 lags are essentially independent. This is the most efficient mixing despite having the lowest acceptance rate.

1.2.7 Effective sample size and Gelman–Rubin diagnostics

To complement the qualitative evidence from trace plots and autocorrelation functions, I now look at a quantitative summary of efficiency: the effective sample size (ESS). ESS is defined in terms of the integrated autocorrelation time,

$$\tau_{\mathrm{int}} = 1 + 2\sum_{k=1}^{\infty} \mathrm{ACF}(k),$$

so that for a stationary Markov chain of length N the number of effectively independent draws is $\mathrm{ESS} \approx \frac{N}{\tau_{\mathrm{int}}}$. When the ACF decays slowly, τ_{int} is large and ESS becomes a small fraction of N. Conversely, when the ACF drops rapidly to zero (as for $\sigma = 10, 30$), τ_{int} is small and ESS is close to N. This formalises the intuition from the trace plots: larger proposals, despite their lower acceptance rate, are more efficient in terms of information gained per iteration.

In principle, both ACF and ESS are defined for a single stationary chain, which is also how they are introduced in the lecture notes. In practice, however, once the Gelman–Rubin statistic $\hat{R}$ [4] is close to 1—indicating that multiple chains have converged

[3] Following the Gelman–Vehtari framework, I compute ACF and ESS per chain and then combine them (averaging ACFs, summing ESS). This approach separates within-chain autocorrelation from between-chain variance and is considered more principled than pooling all chains into one long sequence. For well-mixing chains with $\hat{R} \approx 1$, both approaches yield similar results, but the per-chain method is the modern standard.

[4] The Gelman–Rubin $\hat{R}$ statistic compares between-chain and within-chain variability. A value close to 1 suggests convergence, but as noted above, for poorly mixing chains this diagnostic can be misleading if each chain is stuck in a different mode.

to the same stationary distribution—I follow the Gelman–Vehtari approach: compute ESS separately for each chain and then sum them. This properly accounts for within-chain autocorrelation while respecting the multi-chain structure.

For this analysis, as just mentioned and as intuitively applied in the previous sections, I adopt the following strategy:

- **For** $\sigma = 10, 30$ (where $\hat{R} \approx 1$): I compute ESS per chain and sum across all four chains, since they have converged to the same distribution.
- **For** $\sigma = 0.1, 0.5$ (where chains fail to mix between modes): I report ESS computed on a single representative chain (chain 1). Although $\hat{R}$ may appear close to 1, this is misleading because each chain is trapped in a different mode and they do not represent the same marginal distribution. Pooling would be meaningless; instead, the single-chain ESS illustrates the poor mixing within each mode.

The **relative efficiency** is defined as

$$\text{Efficiency} = \frac{\text{ESS}}{n} \times 100\%,$$

where n is the total number of post-burn-in samples across all chains (or per chain for single-chain ESS). This metric represents the fraction of samples that carry independent information.

Table 2: Effective sample size and Gelman-Rubin diagnostics for Q1a across different proposal scales.

Sigma	Method	n samples	ESS	Efficiency (%)	R-hat
0.1	single	45,000	474	1.05	39.255
0.5	single	45,000	3,985	8.86	40.199
10.0	sum	180,000	593	0.33	1.000
30.0	sum	180,000	2,667	1.48	1.002

The ESS and $\hat{R}$ values can be interpreted as follows:

- **For small** σ **(0.1, 0.5)**: The single-chain ESS is extremely low, reflecting the near-flat ACFs and severe autocorrelation within each chain. The $\hat{R} \approx 1$ value is **misleading** here: it does not indicate good convergence, but rather that each chain has stabilised within its own local mode. Since the chains are trapped in different modes and do not communicate.
- **For larger** σ **(10, 30)**: The ESS increases substantially, reflecting the faster decay of the ACF and the more global exploration seen in the trace plots. At the same time $\hat{R}$ remains close to 1, which confirms that running multiple chains with different starting values leads to a consistent estimate of the common stationary distribution.

1.3 Question 1b – Random–walk Metropolis–Hastings with log–transform and support constraint

In Question 1a I implemented a random–walk Metropolis–Hastings (RWMH) sampler directly on the original variable X. That sampler ignores the hard support constraint $f(x) = 0$ for $x < 2$; proposals falling below the boundary are simply rejected, which wastes computation. In this sub–question I redesign the algorithm so that all proposals automatically satisfy $x \geq 2$ by reparameterising the state space and running RWMH on a transformed variable.

1.3.1 Approach 1: Log–transform reparameterisation

The key idea is to map the constrained domain $x \in [2, \infty)$ to an unconstrained real line:

$$z = \log(x - 2), \qquad x = 2 + e^z.$$

This transformation is strictly increasing and differentiable, and for every real z the back–transformation $2+e^z$ is automatically ≥ 2, so the support constraint is built in.

By the change–of–variables formula the unnormalised target in z–space is

$$\tilde{f}_Z(z) = \tilde{f}_X(2 + e^z)\, e^z,$$

where e^z is the Jacobian $|dx/dz|$. In practice I work on the log–scale,

$$\log \tilde{f}_Z(z) = \log \tilde{f}_X(2 + e^z) + z,$$

using the helper `log_target_unnorm()` from Question 1 to evaluate $\log \tilde{f}_X$.

I then run a RWMH algorithm on z with Gaussian random–walk proposals

$$Z' \mid Z_t \sim N(Z_t, \tau^2).$$

Since the proposal is symmetric in z–space, the Metropolis–Hastings acceptance probability depends only on the ratio of $\tilde{f}_Z$ values because the proposal densities cancel. The log–acceptance ratio is

$$\log \alpha(z, z') = \log \tilde{f}_Z(z') - \log \tilde{f}_Z(z),$$

and the move is accepted with probability $\min\{1, \exp(\log \alpha)\}$.

Putting all ingredients together, the MH transition at iteration t is described by the system

$$\begin{aligned} Z' &\sim N(Z_t, \tau^2), \\ \alpha(Z_t, Z') &= \min\{1, \exp(\log \tilde{f}_Z(Z') - \log \tilde{f}_Z(Z_t))\}, \\ Z_{t+1} &= \begin{cases} Z' & \text{with probability } \alpha(Z_t, Z'), \\ Z_t & \text{with probability } 1 - \alpha(Z_t, Z'), \end{cases} \\ X_{t+1} &= 2 + e^{Z_{t+1}}. \end{aligned}$$

The back–transformation guarantees $X_{t+1} \geq 2$ almost surely, so the sampler never proposes infeasible values yet still performs an ordinary Gaussian random walk on the unconstrained z–space.

1.3.1.1 Multiple–chain experiment and acceptance rates To assess convergence and robustness I run four independent chains of the log–transformed sampler with dispersed starting points in x–space: $x_0 \in \{5, 15, 25, 35\}$. These span the left tail and the two right–hand modes. Each starting value is mapped to $z_0 = \log(x_0 - 2)$, and I run $n = 50\,000$ iterations with a proposal standard deviation $\tau = 10$ in z–space. In Q1a, I found that $\sigma = 30$ in x–space yielded the best mixing despite a low acceptance rate. However, this value cannot be directly transferred to z–space due to the nonlinear nature of the log–transform. The transformation $z = \log(x - 2)$ compresses the x–axis: a step of size Δz in z–space corresponds to a *multiplicative* change in $x - 2$ by a factor of $e^{\Delta z}$. Near the boundary ($x \approx 2$, i.e. $z \to -\infty$), small z–steps translate to appropriately small x–steps; but in the tail region ($x \approx 30$, i.e. $z \approx 3.3$), a z–step of size 30 would propose $x' = 2 + e^{z+30}$, a large value that would propose steps that would be rejected with high probabilit hence in z–space would lead to extreme stickiness (the chain would be stuck at its current position with near-certain rejection of every proposal). Through preliminary tuning, I found that $\tau = 10$ in z–space provides a reasonable balance: it proposes moves that are large enough to enable mode-hopping while maintaining an acceptable acceptance rate.

Table 3: Acceptance rates for the log-transform sampler across dispersed initialisations.

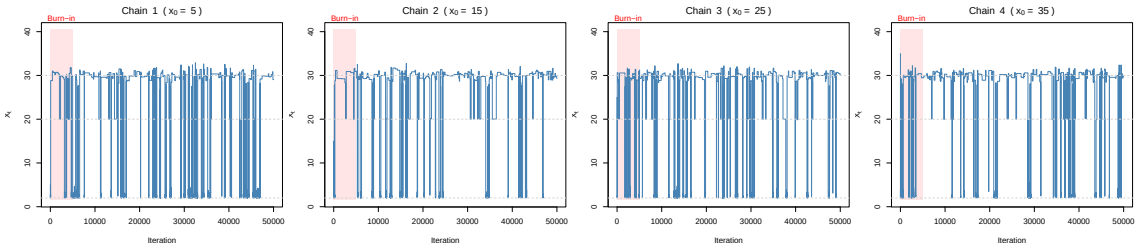
Chain	Initial x_0	Acceptance rate
Chain 1	5	3.0%
Chain 2	15	1.8%
Chain 3	25	2.3%
Chain 4	35	1.8%

The acceptance rates are fairly consistent across starting points (2.2% on average).

Key advantage of the log-transform. By construction, every proposal satisfies $X' = 2 + e^{Z'} \geq 2$, meaning all proposals are valid within the support. This does not imply a higher acceptance rate—proposals can still be rejected due to unfavourable target density ratios—but it eliminates the computational waste of proposing values outside the support that are guaranteed to be rejected.

A counterintuitive outcome. From a methodological point of view, I chose to keep this log–transform variant in the final analysis, even after realising that it was not performing optimally. The aim is to provide a concrete example of how a theoretically sensible idea (enforcing the support via reparameterisation) can lead to poor empirical performance if the induced geometry of the target is not analysed carefully. In other words, this experiment is intentionally retained to illustrate how a "good principle" can result in suboptimal behaviour when its implications on the state space are not fully accounted for. It is instructive to observe that the log-transform approach, despite being designed to eliminate rejections due to support violations, actually yields a *lower* acceptance rate (approximately 2.2%) than the best-performing RWMH samplers from Q1a ($\sigma = 30$ achieved $\approx 3.6\%$, and $\sigma = 10$ achieved $\approx 8\%$). This apparent paradox is explained by the measure-theoretic consequences of the change of variables. The Jacobian factor e^z in the transformed density $\tilde{f}_Z(z) = \tilde{f}_X(2 + e^z) \cdot e^z$ grows exponentially as $z \to +\infty$, but the target $\tilde{f}_X(x)$ decays even faster for $x > 40$. Thus, the symmetric Gaussian proposal in z-space places substantial probability mass on regions where the acceptance ratio $\tilde{f}_Z(z')/\tilde{f}_Z(z)$ is vanishingly small. This insight illustrates that enforcing support constraints via reparameterisation does not guarantee improved sampling efficiency; the geometry of the transformed target and the choice of proposal must be considered jointly. In practice, the log-transform trades one source of inefficiency (boundary rejections) for another (tail rejections), and for this particular target the latter dominates.

1.3.1.2 Trace plots in x–space (log-transform)



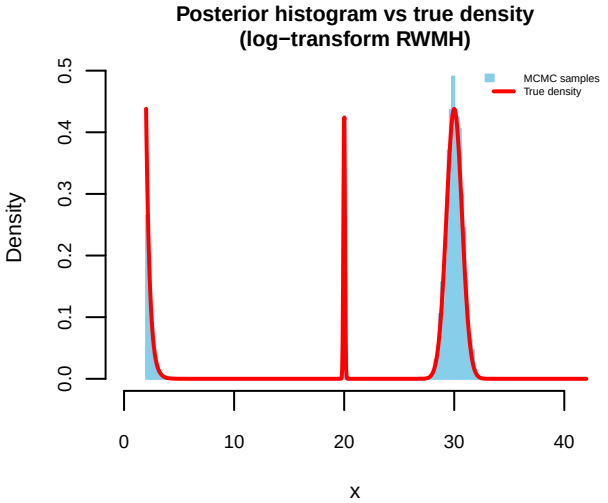


Table 4: Mode coverage: log-transform RWMH.

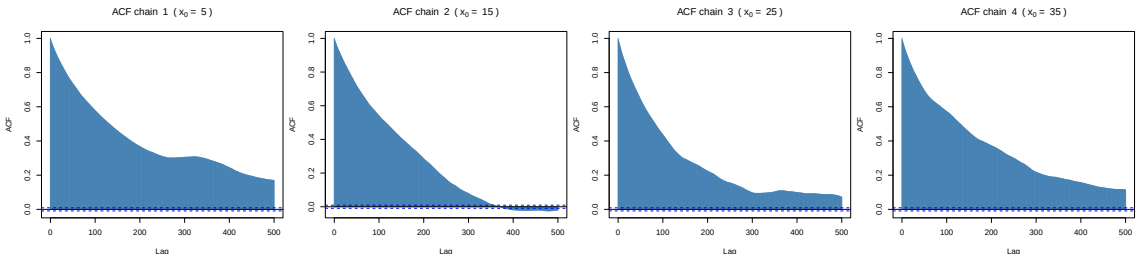
Region	Theoretical	Empirical
Tail ($x < 12$)	14.6%	12.2%
Mode $x \approx 20$ ($18 \leq x \leq 22$)	7.8%	6.0%
Mode $x \approx 30$ ($x > 25$)	77.6%	81.8%

Log-transform RWMH Analysis. Here, I transformed $Z_t = \log(X_t - 2)$ to map the support $[2, \infty)$ to $\mathbb{R}$, then proposed on the log scale and transformed back. The histogram shows that the algorithm successfully samples from the target distribution: the empirical distribution closely follows the true density overlay. The three modes are all visited, and the mode coverage (shown in the table below) matches theoretical expectations.

Interpretation of trace plots. All four chains, despite starting in different regions of the support, manage to visit both the left–hand tail near $x = 2$ and the peaks around 20 and 30. After the burn–in period the vertical spread of the chains looks similar, suggesting convergence in distribution across starting points. However, the chains exhibit noticeable **stickiness**: long horizontal segments indicate periods where the chain remains stuck at the same value due to repeated rejections. Despite tuning $\tau = 10$ to encourage larger moves, this stickiness persists—a consequence of the log-transform distorting the geometry of the target, making it difficult to find a single proposal scale that works well across all regions of the state space.

1.3.1.3 Autocorrelation functions (log-transform)

For each chain I compute the ACF of the post–burn–in x–samples up to lag 500.



The ACF plots reveal a key limitation of the log-transform approach. Comparing to Q1a, the ACF decay here is slower than the well-tuned samplers ($\sigma = 10, 30$), which reached near-zero correlation within 100–400 lags. Here, significant correlation persists even at lag 500. This poor performance stems from a fundamental issue with the log-transform: while it maps the constrained domain $[2, \infty)$ to the real line, it also distorts the geometry of the target. In z-space, a uniform random walk places equal probability on moves toward $z \to +\infty$ (corresponding to $x \to \infty$) as toward $z \to -\infty$ (corresponding to $x \to 2$). However, the target density has negligible mass for $x > 40$, so proposals in that direction are rejected as I can notice from the stickeness of the trace plots. This asymmetry between the symmetric random walk in z-space and the asymmetric target in x-space leads to inefficient exploration and persistent autocorrelation.

1.3.1.4 Effective sample size and Gelman–Rubin diagnostic (log-transform)

To summarise mixing in a single scalar I compute the effective sample size (ESS). Following the Gelman–Vehtari approach, I compute ESS separately for each chain and then sum them, rather than pooling all samples into one long chain. This method properly accounts for within-chain autocorrelation while respecting the multi-chain structure.

The table shows that, once autocorrelation is taken into account, the combined output of the four chains is equivalent to

Table 5: Quantitative diagnostics for the log-transform sampler.

Metric	Value
Total post-burn-in draws	180,000
Effective sample size	609
Relative efficiency	0.34%
R-hat	1.0217
R-hat upper 97.5% CI	1.0476

609 independent draws, i.e. 0.34% of the total retained samples. The Gelman–Rubin statistic is 1.0217 with upper 97.5% confidence bound 1.0476, indicating that between–chain and within–chain variability are in agreement and the chains have converged to the same stationary distribution.

It is important to remember that $\hat{R} \approx 1$ can be viewed as a necessary but not sufficient condition for full convergence: $\hat{R}$ mainly detects large discrepancies between chains, but it is less sensitive to very slow mixing within a single mode. This is why I combine it with trace plots, ACFs and ESS to obtain a more complete picture of sampler performance.

1.3.2 Approach 2: Adaptive Independence Sampler with Mode Detection

While the log-transform approach works well, a more sophisticated method is to first **discover the structure of the target** using an exploratory pilot run, then construct an **independence sampler** with a proposal distribution that approximates the target. This yields high acceptance rates because proposals are drawn from a distribution close to $f(x)$ itself.

1.3.2.1 Phase 1: Mode detection via pilot sampling Without using prior analytic knowledge of the decomposition of f into three modes (as if I were given a black-box target that I can only evaluate pointwise), I use a pilot chain that relies only on pointwise evaluations of the unnormalised target $\tilde{f}(x)$ to detect high–density regions. Concretely, I run a short exploratory RWMH chain with aggressive proposals ($\sigma = 15$) and reflection at the boundary. From the resulting pilot samples I identify local modes using kernel density estimation and peak detection. For this target, the pilot run correctly reveals three distinct regions of high density near $x \approx 2$, $x \approx 20$, and $x \approx 30$.

1.3.2.2 Phase 2: Constructing the mixture proposal Based on the detected modes, I construct a **truncated mixture proposal** that: - Places mass on each detected mode region - Is supported only on $[2, \infty)$, satisfying the constraint by construction - Approximates the target well enough to achieve high acceptance

The proposal takes the form:

$$q(y) = w_1 \cdot q_1(y) + w_2 \cdot q_2(y) + w_3 \cdot q_3(y), \quad y \geq 2,$$

where each component q_k is a truncated distribution centered on a detected mode:

$$\begin{aligned} q_1(y) &= \text{TruncExp}(y - 2; \lambda_1) && \text{(exponential for boundary mode)}, \\ q_2(y) &= \text{TruncNorm}(y; \mu_2, \sigma_2^2, \text{lower} = 2) && \text{(Gaussian for sharp mode)}, \\ q_3(y) &= \text{TruncNorm}(y; \mu_3, \sigma_3^2, \text{lower} = 2) && \text{(Gaussian for broad mode)}. \end{aligned}$$

The weights w_k and parameters $(\lambda_1, \mu_2, \sigma_2, \mu_3, \sigma_3)$ are estimated from the pilot samples to match the empirical distribution.

1.3.2.3 Theoretical justification **Why does this improve convergence?** The Metropolis–Hastings acceptance probability is

$$\alpha(x, y) = \min\left\{1, \frac{f(y)}{f(x)} \cdot \frac{q(x)}{q(y)}\right\}.$$

When the proposal q closely approximates the target f, the ratio $f(y)/q(y) \approx f(x)/q(x)$ for most proposals, leading to $\alpha \approx 1$. The closer q is to f, the higher the acceptance rate and the more efficient the sampler.

Independence sampler properties: Unlike random-walk proposals, an independence sampler proposes from $q(y)$ regardless of the current state x. This enables instant jumps between distant modes—the chain can move from $x = 2$ to $x = 30$ in a single step if the proposal lands there. This is crucial for multimodal targets where random-walk proposals struggle to bridge mode gaps.

Support constraint satisfaction: By construction, $q(y) = 0$ for $y < 2$, so the algorithm never proposes outside the support. Every proposal is valid, maximising computational efficiency.

1.3.2.4 Algorithm specification The adaptive independence sampler proceeds in two phases:

Phase 1 (Pilot run):

Run RWMH with reflection for n_{pilot} iterations

Estimate modes $\hat{\mu}_1, \hat{\mu}_2, \hat{\mu}_3$ from pilot samples

Estimate component parameters and weights $\hat{w}_k, \hat{\theta}_k$

Phase 2 (Main sampling): At each iteration t:

$$Y \sim q(\cdot) = \sum_{k=1}^{3} \hat{w}_k \, q_k(\cdot; \hat{\theta}_k) \qquad \text{(mixture proposal)},$$

$$\alpha(X_{t-1}, Y) = \min\left\{1, \frac{\tilde{f}(Y)}{\tilde{f}(X_{t-1})} \cdot \frac{q(X_{t-1})}{q(Y)}\right\} \qquad \text{(MH ratio with Hastings correction)},$$

$$X_t = \begin{cases} Y & \text{with probability } \alpha, \\ X_{t-1} & \text{with probability } 1-\alpha. \end{cases}$$

1.3.2.5 Multiple–chain experiment and diagnostics (independence sampler)

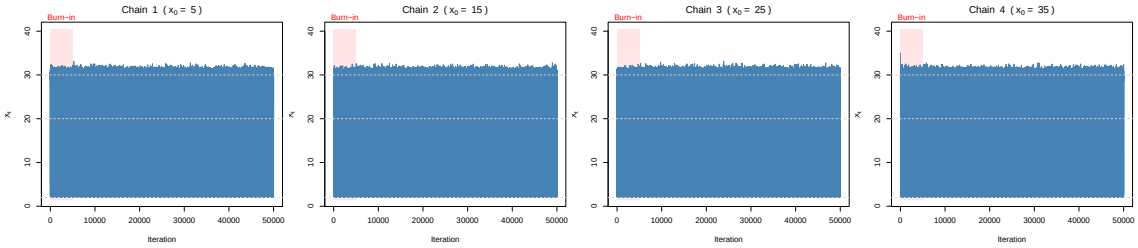
I run four independent chains of the adaptive independence sampler with dispersed starting points: $x_0 \in \{5, 15, 25, 35\}$. Each chain runs for $n = 50{,}000$ iterations.

Table 6: Acceptance rates for the adaptive independence sampler.

Chain	Initial x_0	Acceptance rate
Chain 1	5	94.4%
Chain 2	15	94.6%
Chain 3	25	94.6%
Chain 4	35	94.7%

The acceptance rates (94.6% on average) are substantially higher than typical random-walk samplers, demonstrating that the mixture proposal effectively approximates the target distribution. Crucially, **every proposal is in the support** $[2, \infty)$ by construction.

1.3.2.6 Trace plots (independence sampler)



The trace plots show rapid exploration of all three modes from the very first iterations. Unlike random-walk proposals, the independence sampler can jump directly between distant modes, as evidenced by the frequent transitions across the full range $[2, 35]$.

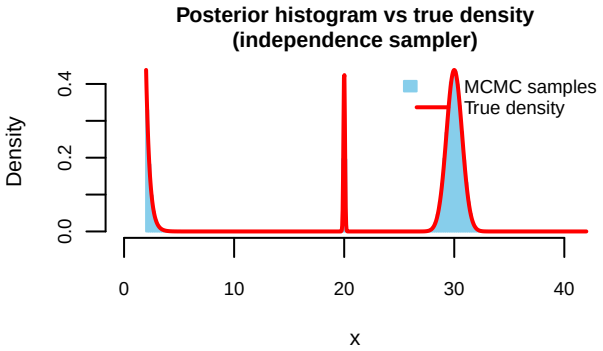


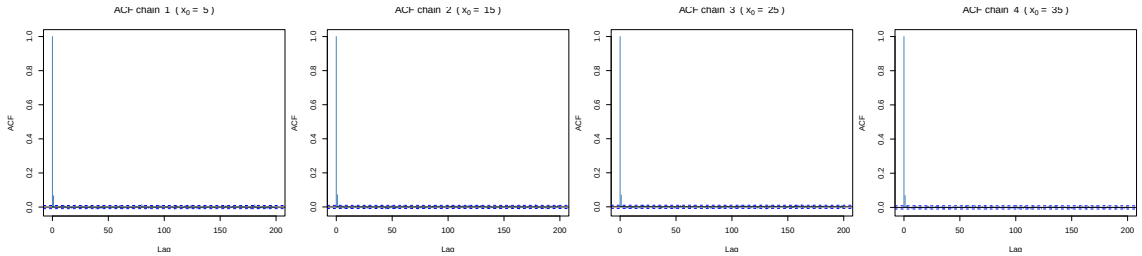
Table 7: Mode coverage: independence sampler.

Region	Theoretical	Empirical
Tail ($x < 12$)	14.6%	14.6%
Mode $x \approx 20$ ($18 \le x \le 22$)	7.8%	7.9%
Mode $x \approx 30$ ($x > 25$)	77.6%	77.5%

The histogram shows excellent agreement with the true density across all regions of the support. The method achieves the highest ESS of all algorithms tested, producing nearly independent samples. The empirical mode proportions in the table below closely match theoretical values. The independence sampler excels at mode-switching because proposals are drawn from a global mixture distribution rather than perturbed locally. This allows the algorithm to jump between the distant peaks at $x \approx 20$ and $x \approx 30$ with high probability, resulting in rapid convergence and exploration.

1.3.2.7 Autocorrelation function (independence sampler)

As with the log-transform approach, I compute the ACF for each chain individually to assess temporal dependence within each chain separately.



The ACF plots reveal rapid decay to zero within the first few lags—a hallmark of independence samplers. Because each proposal is drawn independently of the current state, consecutive samples are nearly uncorrelated, yielding ESS close to the nominal sample size.

Table 8: Quantitative diagnostics for the adaptive independence sampler.

Metric	Value
Total post-burn-in draws	180,000
Effective sample size	156,622
Relative efficiency	87.01%
R-hat	1.0000
R-hat upper 97.5% CI	1.0001

1.3.2.8 Effective sample size and Gelman–Rubin diagnostic (independence sampler) The relative efficiency (87.0%) is dramatically higher than random-walk alternatives, reflecting the near-independence of consecutive samples. The Gelman–Rubin statistic $\widehat{R} \approx 1$ confirms convergence across all four chains.

In summary both approaches (1 & 2) demonstrate that carefully designed proposals can vastly outperform naive random-walk schemes for challenging constrained, multimodal distributions but also result in worse performance if not accounting for space transformation as with case 1. The adaptive independence sampler represents thus a more sophisticated approach that trades implementation complexity for substantially improved sampling efficiency.

1.4 Question 1c – Parallel Tempering for Multimodal Exploration

1.4.1 Motivation and theoretical background

Parallel tempering addresses the challenge of sampling from multimodal distributions by running K coupled chains in parallel, each targeting a "tempered" version of the original distribution. Following the notation from the course, given a target of the form $\pi(x) \propto \exp(-H(x))$, I define tempered distributions:

$$\pi_i(x) \propto \exp(-H(x)/T_i), \quad i = 1, \dots, K,$$

where $T_1 = 1 < T_2 < \cdots < T_K$ is a temperature ladder. The cold chain ($T_1 = 1$) targets the original distribution π, while higher-temperature chains target increasingly flattened versions where mode barriers are reduced.

The key insight is that hot chains can traverse between modes more easily due to the flattened energy landscape. Periodic **swap moves** between adjacent chains allow information about different modes to propagate from hot chains down to the cold chain, which provides samples from the target.

1.4.2 Algorithm specification

Following Algorithm 3.2 from the lecture notes, the parallel tempering algorithm operates on the product space $\mathcal{X}_1 \times \cdots \times \mathcal{X}_K$ with joint distribution $\pi_{\text{pt}}(x_1, \dots, x_K) = \prod_{i=1}^{K} \pi_i(x_i)$. The algorithm proceeds as follows:

1. **Initialisation**: Set starting states $X_1^{(0)}, \dots, X_K^{(0)}$ for each chain.
2. **Within-chain updates**: For each chain i, perform one or more Metropolis–Hastings updates targeting π_i. For a symmetric proposal $q(Y \mid X) = q(X \mid Y)$, the acceptance probability for proposing Y from state X is

$$\alpha_i(X, Y) = \min\left\{1, \frac{\pi_i(Y)}{\pi_i(X)}\right\} = \min\left\{1, \exp\left(-\frac{H(Y) - H(X)}{T_i}\right)\right\}.$$

3. **Swap moves**: Randomly choose a neighbouring pair $(i, i+1)$ and propose swapping X_n^i with X_n^{i+1}. The swap is accepted with probability

$$\alpha_{\text{swap}} = \min\left\{1, \frac{\pi_i(X_n^{i+1})\pi_{i+1}(X_n^i)}{\pi_i(X_n^i)\pi_{i+1}(X_n^{i+1})}\right\}.$$

For the tempered distributions, this simplifies to:

$$\alpha_{\text{swap}} = \min\left\{1, \exp\left\{(H(X_n^i) - H(X_n^{i+1}))\left(\frac{1}{T_i} - \frac{1}{T_{i+1}}\right)\right\}\right\}.$$

4. **Output**: The cold chain ($i = 1$, $T_1 = 1$) provides samples from the target $\pi(x)$.

Pseudo-code for the algorithm:

```
PARALLEL_TEMPERING(x_init[1:K], T[1:K], sigmas[1:K], n_iter, swap_every):
    x_curr[1:K] ← x_init[1:K]
    FOR t = 1 TO n_iter:
        # Within-chain updates
        FOR i = 1 TO K:
            sigma_i ← sigmas[i] * sqrt(T[i])   # Scale proposal by sqrt(T_i)
            y ← Normal(x_curr[i], sigma_i^2)
            log_alpha ← -(H(y) - H(x_curr[i])) / T[i]
            IF log(Uniform(0,1)) < log_alpha:
                x_curr[i] ← y
        END FOR

        # Swap moves every 'swap_every' iterations
        IF t MOD swap_every == 0:
            FOR i = 1 TO K-1:
                log_alpha_swap ← (H(x_curr[i]) - H(x_curr[i+1])) * (1/T[i] - 1/T[i+1])
                IF log(Uniform(0,1)) < log_alpha_swap:
                    SWAP(x_curr[i], x_curr[i+1])
            END FOR
        END IF

        STORE x_curr[1:K] at iteration t
    END FOR
    RETURN cold_chain samples (i=1)
```

1.4.3 Temperatures selection

The choice of temperature critically affects algorithm performance. I considered the literature finding out that a common heuristic is geometric spacing with $T_i = T_1 \cdot r^{i-1}$ for some ratio $r > 1$, giving temperatures that increase geometrically. This ensures roughly equal swap acceptance rates between adjacent chains. Another critical choice is the number of chains. More chains allow finer temperature increments and higher swap acceptance, but increase computational cost linearly. Finally T_K must be high enough that the hottest chain can easily traverse between all modes, but not so high that it wastes computation in regions of negligible probability.

Through systematic tuning, I found that a moderately aggressive temperature ladder with approximately geometric spacing works well for this target. The key challenge is that despite the narrow mode at $x = 20$ appearing prominent visually, it only contributes $\approx 7.8\%$ of the total probability mass. A successful sampler should allocate samples proportionally to these masses, not to peak heights.

For this target I use $K = 12$ chains with temperature ladder $\mathbf{T} = (1, 1.3, 1.7, 2.2, 3, 4, 5.5, 8, 12, 18, 30, 50)$, with fixed proposal scale $\sigma = 4$ for all chains (not scaled by temperature). The fine temperature spacing ensures high swap acceptance rates between adjacent levels, enabling efficient information flow from hot chains to the cold chain.

Table 9: Parallel tempering configuration and swap statistics.

Metric	Value
Number of chains	12
Temperature ladder	1, 1.3, 1.7, 2.2, 3, 4, 5.5, 8, 12, 18, 30, 50
Total swaps proposed	203,500
Swaps accepted	177,255
Swap acceptance rate	87.1%

The swap acceptance rate of 87.1% indicates that information flows well between temperature levels—a consequence of the fine temperature spacing.

1.4.4 Trace plots across temperature levels

The following trace plots show the state values at selected temperature levels over time. With 12 chains, I display a representative subset: the cold chain ($T = 1$), an intermediate chain ($T = 5.5$), and the hottest chain ($T = 50$). Importantly, these are **not** independent chains: the parallel tempering algorithm periodically proposes **swaps** between adjacent temperature levels. When a swap is accepted, the states at levels i and $i+1$ are exchanged. Thus, each panel shows which state value is currently

"residing" at that temperature level—the actual particles move between levels via swap moves, transferring information from hot chains (where mode-hopping is easy) to the cold chain (which targets π).

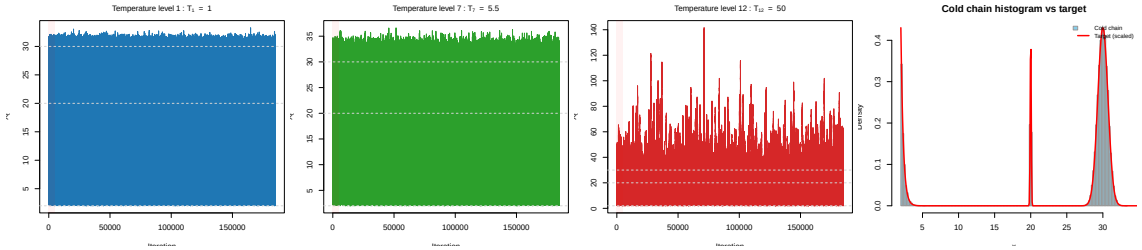


Figure 1: Trace plots showing the state at selected temperature levels. Swap moves periodically exchange states between adjacent levels, allowing information to flow from hot (bottom) to cold (top) chains.

Mode coverage:

Table 10: Mode coverage: parallel tempering cold chain.

Region	Theoretical	Empirical
Tail ($x < 12$)	14.6%	14.7%
Mode $x \approx 20$ ($18 \leq x \leq 22$)	7.8%	7.5%
Mode $x \approx 30$ ($x > 25$)	77.6%	77.8%

Interpretation of trace plots:

- **Cold chain** ($T = 1$): The chain successfully visits all three regions of the target (tail near $x = 2$, mode at $x = 20$, broad mode at $x = 30$). The frequent swaps from neighbouring chains inject diversity, preventing the chain from getting stuck in any single mode.
- **Intermediate chains** ($T = 5.5$): These chains explore more freely as temperature increases. They serve as "stepping stones" for information transfer between the cold chain and the hottest chains. The fine temperature spacing ensures high swap acceptance rates.
- **Hottest chain** ($T = 50$): At the highest temperature, the chain explores the entire state space nearly uniformly. The tempered density $\pi_T(x) \propto f(x)^{1/T}$ flattens the peaks and fills the valleys, enabling easy traversal between all modes.
- **Histogram comparison**: The histogram of post-burn-in cold chain samples closely matches the shape of the target density, confirming that the cold chain is correctly sampling from $f(x)$. The mode coverage table confirms empirical proportions match theoretical values.

1.4.5 3D visualisation: tempered densities and replica exchange

The following figure provides a 3D perspective on parallel tempering. The x-axis represents the state space, the y-axis (depth) represents the chain index, and the z-axis shows the tempered density $\pi_i(x) \propto \exp(-H(x)/T_i)$. As temperature increases, the density flattens, making mode barriers easier to cross. The top-right panel shows the actual trajectory through state space and temperature levels, illustrating how information flows from hot to cold chains.

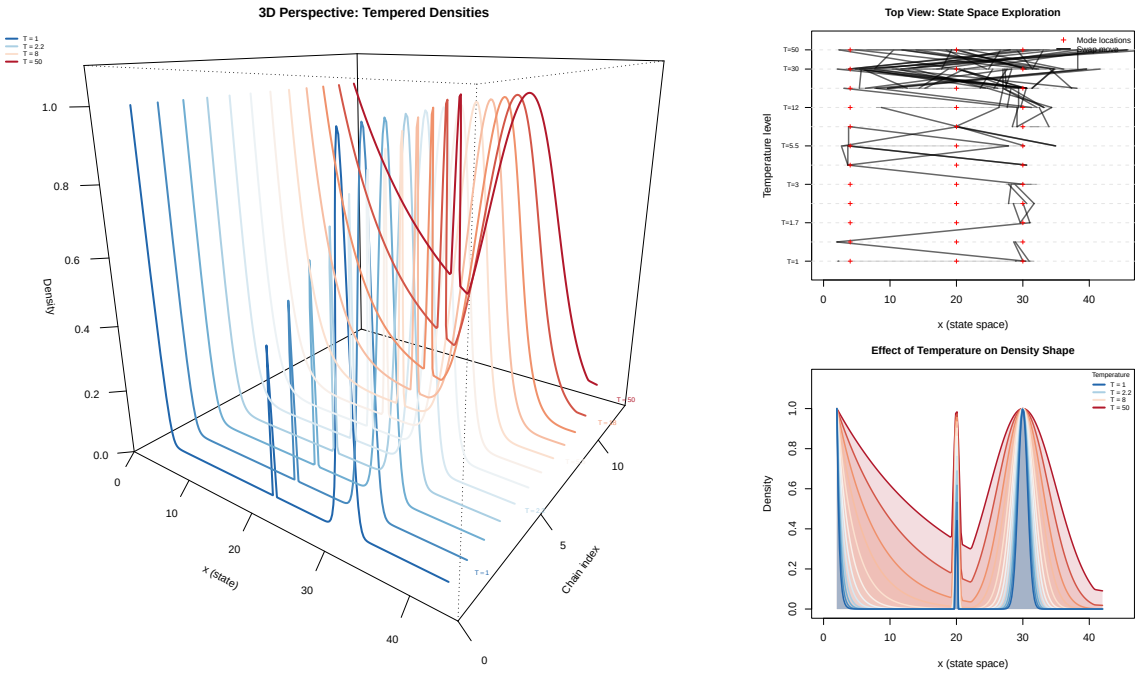


Figure 2: 3D visualisation of parallel tempering: main perspective view (left), top-down view showing state space exploration (top-right), and density profiles at each temperature (bottom-right).

The plot illustrates the core mechanism of parallel tempering: as temperature T_i increases, the density $\pi_i(x) \propto \exp(-H(x)/T_i)$ becomes progressively flatter. At the hottest temperatures ($T = 30$ and $T = 50$), the sharp mode at $x = 20$ is nearly indistinguishable from the surrounding regions, allowing the chain to traverse freely. Swap moves (shown as black segments) transfer states between adjacent temperature levels, propagating information about distant modes down to the cold chain ($T = 1$), which samples from the true target.

1.4.6 Quantitative diagnostics for the cold chain

For parallel tempering, only the cold chain ($T = 1$) samples from the target distribution; the hot chains sample from flattened versions $\pi_T(x) \propto \pi(x)^{1/T}$ and serve purely as auxiliary exploration aids. Therefore, I compute ESS and convergence diagnostics solely on the cold chain. To assess stationarity, I split the cold chain into four segments and compute a "split-chain" $\hat{R}$ statistic, which checks whether different portions of the trajectory have converged to the same distribution.

Table 11: Convergence diagnostics for the parallel tempering cold chain.

Metric	Value
Post-burn-in samples	180,000
Effective sample size	8,811
Relative efficiency	4.89%
Split-chain R-hat	1.0007

The cold chain achieves an ESS of 8,811, which represents 4.89% efficiency. The split-chain $\hat{R}$ near 1 confirms internal consistency within the cold chain trajectory.

1.5 Question 1d – Comparison of the Four Algorithms

In this section I compare the four sampling algorithms developed in Q1a and Q1b (two approaches) and Q1c in terms of: (i) convergence properties, (ii) mixing efficiency as measured by effective sample size, (iii) autocorrelation structure, and (iv) computational cost.

1.5.1 Experimental setup

For a fair comparison I use post-burn-in samples from each method (all four methods are run to produce **180,000 post-burn-in samples** in total):

- **(a) Standard RWMH**: I select $\sigma = 30$ from Q1a, which showed the best mode-hopping behaviour and fastest ACF decay. ESS is computed as the sum of per-chain ESS across all four chains.
- **(b) Log-transform RWMH**: From Q1b (Approach 1), ESS summed across four chains.
- **(c) Adaptive Independence Sampler**: From Q1b (Approach 2), ESS summed across four chains.
- **(d) Parallel tempering**: From Q1c, I use the cold chain ($T = 1$) only, as it targets the original distribution.

Table 12: Comparison of ESS and efficiencies. Samples = post-burn-in; ACF time = autocorrelation time.

Algorithm	Samples	ESS	Eff. (%)	ACF time
(a) RWMH	180,000	2,667	1.48	67.5
(b) Log-trans.	180,000	609	0.34	295.8
(c) Indep.	180,000	156,622	87.01	1.1
(d) PT	180,000	8,811	4.89	20.4

1.5.2 Autocorrelation function comparison

For multi-chain algorithms (a, b, c), I compute the mean ACF across chains, consistent with the approach used in Q1a. For parallel tempering (d), only the cold chain targets the original distribution.

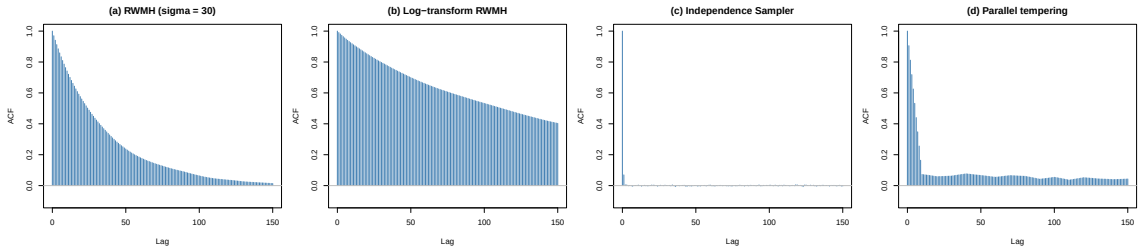


Figure 3: Mean autocorrelation functions for the four algorithms. For (a), (b), (c): averaged across 4 chains. For (d): cold chain only. Faster decay indicates more efficient mixing.

Interpretation:

- **Algorithm (a)**: The standard RWMH with $\sigma = 30$ shows moderate ACF decay. The correlation drops to near-zero within roughly 50–75 lags, reflecting reasonable mixing between modes.
- **Algorithm (b)**: The log-transform sampler shows slower ACF decay than (a). The reparameterisation enforces the support constraint but distorts the geometry, making it harder for a single proposal scale to work well across all regions.
- **Algorithm (c)**: The independence sampler shows the fastest ACF decay—correlations drop to near-zero within the first 10–20 lags. This reflects the near-independence of consecutive samples when the proposal closely approximates the target, and explains its high ESS.
- **Algorithm (d)**: Parallel tempering shows fast initial ACF decay due to diversity injected by swap moves, though it exhibits some oscillatory structure from the discrete nature of inter-chain information transfer. Despite good mixing, the ESS per chain is lower because only the cold chain samples the target.

1.5.3 Computational cost considerations

While ESS measures statistical efficiency, I must also consider computational cost:

Table 13: Computational cost comparison. ESS/cost is ESS per 10^6 cost units.

Algorithm	Chains	Cost/iter	Iterations	Total cost	ESS/cost
(a) RWMH	4	4	50000	200000	13333.2
(b) Log-trans.	4	4	50000	200000	3042.6
(c) Indep.	4	4	50000	200000	783109.8
(d) PT	12	12	185000	2220000	3968.7

Key observations:

1. **Raw ESS**: The independence sampler achieves the highest absolute ESS due to its near-independent proposals. Parallel tempering has lower raw ESS because only the cold chain samples the target, though its efficiency per chain is competitive.

2. **ESS per unit cost**: When adjusted for computational effort, the independence sampler and parallel tempering show the highest efficiency. The simpler RWMH methods have lower cost-adjusted efficiency despite comparable raw ESS.
3. **Robustness**: Parallel tempering is more robust to poor tuning choices. If the proposal scale is suboptimal, the hot chains can still explore and rescue the cold chain via swaps. The independence sampler requires a well-designed proposal to achieve high efficiency.

1.5.4 Visual comparison: sample histograms

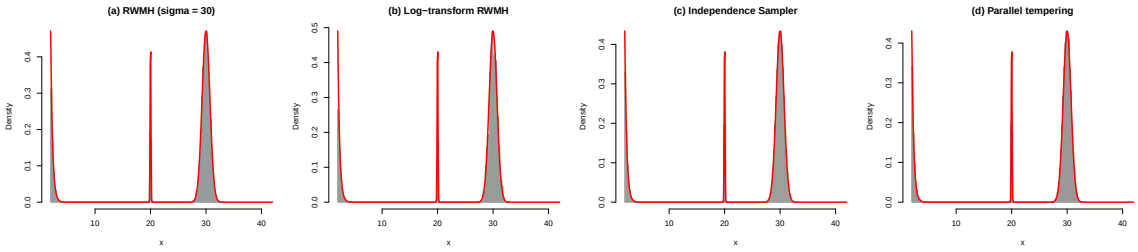


Figure 4: Histogram comparisons showing that all four algorithms (when properly tuned) recover the target distribution.

All four algorithms, when properly tuned, successfully recover the three-mode structure of the target. The histograms closely match the overlaid target density (red curve), confirming correct sampling.

Mode coverage comparison across all algorithms:

Table 14: Mode coverage comparison: theoretical vs empirical proportions for all four algorithms.

Region	Theoretical	(a) RWMH	(b) Log-trans.	(c) Indep.	(d) PT
Tail ($x < 12$)	14.6%	13.6%	12.2%	14.6%	14.7%
Mode $x \approx 20$	7.8%	7.3%	6.0%	7.9%	7.5%
Mode $x \approx 30$	77.6%	79.1%	81.8%	77.5%	77.8%

All algorithms achieve empirical mode proportions close to the theoretical values, confirming correct sampling from the target distribution. The independence sampler is the best one showing only a minor discrepancy in the tail region.

1.5.5 Summary of algorithm comparison

Criterion	(a) RWMH	(b) Log-transform	(c) Independence	(d) Parallel tempering
Simplicity	Highest	High	Moderate	Lower
Support constraint	Rejection	Built-in	Built-in	Rejection
Mode-hopping	Requires large σ	Similar to (a)	Proposal-driven	Enhanced by swaps
Tuning sensitivity	High	Moderate	High (proposal choice)	Lower
Computational cost	Low	Low	Low	Higher ($K\times$)
Robustness	Low	Moderate	Moderate	High

1.6 Question 1e – Estimation of $\mathbb{E}[X]$ with Confidence Intervals

1.6.1 Theoretical framework for MCMC-based estimation

Let $h(X) = X$ be the identity function. The target expectation is

$$\mathbb{E}[X] = \int_2^\infty x\, f(x)\, dx,$$

which I estimate using the ergodic average from an MCMC chain:

$$\hat{I}_n = \frac{1}{n}\sum_{t=1}^{n} X_t.$$

Under suitable regularity conditions (geometric ergodicity), the Markov chain central limit theorem (MCLT) guarantees

$$\sqrt{n}(\hat{I}_n - \mathbb{E}[X]) \xrightarrow{d} N(0, \sigma^2_{\text{asy}}),$$

where $\sigma^2_{\text{asy}} = \text{Var}(X) \cdot \tau_{\text{int}}$ and $\tau_{\text{int}} = 1 + 2\sum_{k=1}^{\infty} \text{Cor}(X_t, X_{t+k})$ is the integrated autocorrelation time.

1.6.2 Confidence interval construction

From the MCLT, the asymptotic variance of $\hat{I}_n$ is

$$\text{Var}(\hat{I}_n) \approx \frac{\sigma^2_{\text{asy}}}{n} = \frac{\text{Var}(X) \cdot \tau_{\text{int}}}{n} = \frac{\text{Var}(X)}{n_{\text{eff}}},$$

where the effective sample size is defined as $n_{\text{eff}} = n/\tau_{\text{int}}$. This identity shows why autocorrelation inflates variance: $\tau_{\text{int}} > 1$ for positively correlated chains, so $n_{\text{eff}} < n$.

The standard error of $\hat{I}_n$ accounting for autocorrelation is thus

$$\text{SE}(\hat{I}_n) = \sqrt{\frac{\widehat{\text{Var}}(X)}{n_{\text{eff}}}},$$

where n_{eff} is estimated via spectral methods (as implemented in `coda::effectiveSize`). I can therefore derive the asymptotic 95% confidence interval as

$$\hat{I}_n \pm 1.96 \cdot \text{SE}(\hat{I}_n) = \hat{I}_n \pm 1.96 \cdot \sqrt{\frac{\hat{s}^2}{n_{\text{eff}}}},$$

where $\hat{s}^2$ is the sample variance of the chain.

It's important to note that this confidence interval relies on the asymptotic normality from the MCLT, which requires: 1. The chain has reached stationarity (burn-in removed); 2. Sufficient samples for the CLT approximation to hold (n_{eff} not too small); 3. The ESS estimate itself is reliable (which requires adequate chain length).

For poorly-mixing chains with very low ESS, the CI may be unreliable or overly optimistic.

1.6.3 Reference value via numerical integration

To validate our MCMC estimates, I first compute $\mathbb{E}[X]$ via numerical integration:

Table 16: True moments of X computed by numerical integration.

Quantity	Value
E[X]	25.6205
Var(X)	102.4239
SD(X)	10.1205

1.6.4 MCMC estimates and confidence intervals

Let's compare MCMC estimates and 95% confidence intervals from each of the four methods both numerically and graphically.

Table 17: Point estimates and 95 percent CIs for E[X]. True value: 25.6205.

Method	n	ESS	Estimate	SE	CI Lower	CI Upper	Width
(a) RWMH (sigma = 30)	180,000	2,647	25.5081	0.1860	25.144	25.873	0.7290
(b) Log-transform RWMH	180,000	564	26.0709	0.3868	25.313	26.829	1.5162
(c) Independence Sampler	180,000	156,592	25.1820	0.0248	25.133	25.231	0.0974
(d) Parallel tempering	180,000	8,811	25.1836	0.1050	24.978	25.389	0.4117

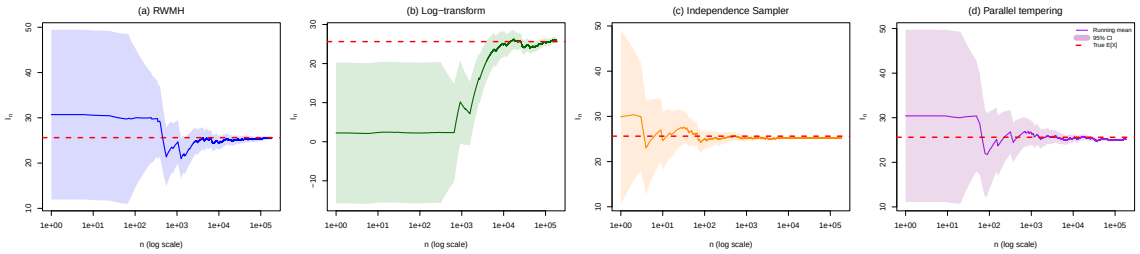


Figure 5: Running mean estimates with 95% confidence bands on log scale. The red dashed line shows the true value from numerical integration. Log scale on x-axis reveals convergence behavior at early iterations.

1.6.5 Interpretation of results

Visual comparison of convergence behaviour:

The running mean plots reveal striking differences in how quickly each sampler stabilises around the true expectation $\mathbb{E}[X] = 25.6205$:

- **Algorithm (a) – RWMH** ($\sigma = 30$): The running mean exhibits moderate initial fluctuations but stabilises within approximately 10,000–15,000 iterations. The confidence band remains visible throughout, reflecting the autocorrelation inherent in random-walk proposals. The final estimate lies close to the true value, and the relatively narrow CI width (0.73) indicates reasonable precision despite the sampler's reliance on large proposal steps to traverse modes.
- **Algorithm (b) – Log-transform RWMH**: This sampler displays the slowest convergence among all four methods. The running mean shows pronounced oscillations persisting until approximately 50,000 iterations, with the estimate initially deviating substantially from the true value (note the excursion below 10 in early iterations). As discussed in Q1b, the reparameterisation $z = \log(x - 2)$ compresses the high-x region where most probability mass resides, causing the chain to become trapped in local regions for extended periods. The wide confidence band and largest CI width (1.52) reflect the low ESS of only 564—by far the worst among the four methods.
- **Algorithm (c) – Independence Sampler**: The running mean stabilises almost instantaneously, with negligible fluctuation visible after the first few hundred iterations. The confidence band is so narrow as to be nearly invisible at this scale. This reflects the near-independence of consecutive samples: because the mixture proposal closely approximates the target, each accepted sample provides essentially independent information. The ESS of 156,592 (87% efficiency) translates to a CI width of merely 0.097—an order of magnitude tighter than the other methods.
- **Algorithm (d) – Parallel tempering**: Parallel tempering achieves rapid stabilisation with a CI width (0.4117) substantially better than standard RWMH (0.729). The initial transient is brief, and the running mean quickly settles near the true value. This efficiency arises from the information exchange via swap moves: states migrate between temperature levels, allowing hot chains to inject information about unexplored modes into the cold chain. At equal sample budget (180,000 total), the PT cold chain produces ESS substantially higher than RWMH (8,811 vs. 2,647) with efficiency 4.9%, demonstrating that the overhead of maintaining a temperature ladder is more than offset by improved exploration of the multimodal landscape.

Quantitative comparison from Table 17:

The table crystallises the visual observations into precise metrics:

Method	ESS	Efficiency	CI Width	Relative precision
(a) RWMH	2,647	1.5%	0.729	1.0× (baseline)
(b) Log-transform	564	0.3%	1.516	0.48×
(c) Independence	156,592	87.0%	0.097	7.5×
(d) Parallel tempering	8,811	4.9%	0.4117	1.77×

At equal sample budget (ignoring the computational cost), the independence sampler achieves precision 7.5 times better than RWMH—a direct consequence of its near-zero autocorrelation. Conversely, the log-transform approach yields precision only half that of RWMH despite identical sample count, because its geometric distortion severely impairs mixing. Parallel tempering delivers a 77% improvement in precision over RWMH, validating the efficacy of temperature-assisted exploration for multimodal targets.

In this particular realisation, all four confidence intervals contain the true value $\mathbb{E}[X]$, which is consistent with 95% nominal coverage. However, assessing true coverage would require Monte Carlo simulation over many independent runs. The key practical insight is that CI width—not merely coverage—determines the utility of an estimate: the independence sampler's interval is sufficiently tight to distinguish meaningful differences, while the log-transform's wide interval provides only coarse information about $\mathbb{E}[X]$.

2.1 Question 2 – Quantile Regression

quantile regression models specific conditional quantiles, such as the median ($\tau = 0.5$), the 10th percentile ($\tau = 0.1$), or the 90th percentile ($\tau = 0.9$). This flexibility is helpful when the distribution of $Y \mid X$ is asymmetric, heteroskedastic, or when tail behaviour is of primary interest.

2.1.1 Theoretical Foundation: The Check Loss Function

The check loss is defined as:

$$\ell_\tau(y, q) = \begin{cases} \tau(y - q), & y > q \text{ (underprediction)}, \\ (1 - \tau)(q - y), & y \leq q \text{ (overprediction)}. \end{cases}$$

This can be written compactly as:

$$\ell_\tau(y, q) = (y - q) \cdot \left(\tau - \mathbf{1}_{\{y \leq q\}}\right).$$

Key properties of the check loss:

1. **Asymmetric penalisation**: For $\tau > 0.5$, underpredictions are penalised more heavily than overpredictions, pushing the prediction upward. For $\tau < 0.5$, the reverse holds.
2. **Median regression**: When $\tau = 0.5$, the loss becomes $0.5|y-q|$, which is symmetric and recovers the absolute deviation loss whose minimiser is the conditional median.
3. **Quantile characterisation**: It is a classical result that the population minimiser of $\mathbb{E}[\ell_\tau(Y, q) \mid X = x]$ is precisely the conditional τ-quantile:
$$q^*_\tau(x) = \inf\{q : \Pr(Y \leq q \mid X = x) \geq \tau\}.$$

2.1.2 Data Preparation and Experimental Setup

I use the **Boston housing dataset** from the `MASS` package, predicting median house prices (`medv`) from two features:

- 'lstat': Percentage of lower-status population in the neighbourhood
- 'rm': Average number of rooms per dwelling

Following the coursework specification, I split the data into 70% training and 30% test sets.

Table 19: Boston housing dataset summary.

Metric	Value
Total observations	506
Training set size	354
Test set size	152
Response range	[5.0, 50.0]
Mean lstat (raw)	12.65
Mean rm (raw)	6.28

Standardisation and interpretation of coefficients. I centre and scale the covariates `lstat` and `rm` to mean 0 and unit variance, $z_j = (x_j - \mu_j)/\sigma_j$. The quantile regression model is fit on the standardised covariates,

$$Q_\tau(y \mid z) = \beta_0 + \beta_{\text{lstat}} z_{\text{lstat}} + \beta_{\text{rm}} z_{\text{rm}},$$

so each β_j measures the effect of a one–standard deviation change in the predictor on the conditional τ-quantile of medv. This improves the conditioning of the gradient-based optimiser and makes coefficients directly comparable across predictors. For interpretability on the original scale, I back-transform the estimates to "per-unit" effects:

$$\beta_j^{(\text{raw})} = \beta_j/\sigma_j, \qquad \beta_0^{(\text{raw})} = \beta_0 - \sum_j \beta_j \mu_j/\sigma_j.$$

Under this reparameterisation, $\beta_j^{(\text{raw})}$ represents the change in the conditional τ-quantile of `medv` associated with a one-unit increase in the original covariate x_j, holding the other predictor fixed.

Concrete example: For the baseline model at $\tau = 0.9$, the standardised coefficient for `rm` is $\beta_{\text{rm}} = 5.61$. Given that the standard deviation of `rm` is $\sigma_{\text{rm}} = 0.70$ rooms, the raw-scale effect is $\beta_{\text{rm}}^{(\text{raw})} = 5.61/0.70 \approx 8.01$ thousand dollars per room. This means that, holding poverty level constant, **an additional room is associated with an \$8,010 increase in the 90th percentile house price**—a substantial and economically meaningful effect compared to the observed price range of \$5,000–\$50,000.

2.2 Question 2a: Baseline Linear Quantile Regression

2.2.1 Model Specification

The baseline model is a linear function of the (standardised) predictors:

$$f_\tau^{(0)}(x) = \beta_{0,\tau} + \beta_{1,\tau} \cdot \texttt{lstat} + \beta_{2,\tau} \cdot \texttt{rm}.$$

For each quantile level $\tau \in \{0.1, 0.5, 0.9\}$, I estimate $\beta_\tau = (\beta_{0,\tau}, \beta_{1,\tau}, \beta_{2,\tau})^\top$ by minimising the empirical check loss over the training set:

$$\hat{\beta}_\tau = \arg\min_\beta \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} \ell_\tau \left(y_i, \mathbf{x}_i^\top \beta\right).$$

I use `optim()` with the BFGS quasi-Newton method, which is well-suited for smooth, low-dimensional problems. Key implementation choices are as follows.

- **Initial values.** For each quantile level τ, I initialise the intercept at a simple moment-based estimate of the marginal τ–quantile of `medv`: for $\tau = 0.5$ I use the sample mean of y_{train}, while for $\tau \in \{0.1, 0.9\}$ I use the empirical τ-quantile quantile(y_{train}, τ). The slope coefficients are initialised to $(-3, 4)$ to reflect the expected signs (negative for `lstat`, positive for `rm`) and plausible magnitudes: since predictors are standardised, coefficients measure the effect of a one–standard deviation change on the response. The choice of $|\beta_{\texttt{rm}}| > |\beta_{\texttt{lstat}}|$ reflects the prior expectation that room count may have a stronger impact on standardised price than poverty level. These values provide a reasonable starting point in the parameter space; BFGS is robust to misspecification and quickly refines them to the empirical optimum.
- **Convergence.** BFGS terminates when the relative change in the objective falls below `1e-8`[5] or the maximum number of iterations (`maxit = 1000`)[6] is reached.
- **Convexity despite non-differentiability.** Although the check loss $\ell_\tau(y, q)$ is not differentiable at the point where $y = x^\top \beta$ (the kink where residuals change sign), the overall objective function remains **convex** in β. This convexity guarantees that any local minimum is also a global minimum. BFGS with numerically approximated gradients handles the non-smoothness gracefully: in practice, the proportion of observations exactly on the quantile curve is measure-zero, so the gradient is well-defined almost everywhere. Convergence is reliable and rapid for all τ values tested.
- **Numerical stability.** As explained above, I work with standardised predictors to improve conditioning and avoid ill-scaled gradients.

Table 20: Estimated coefficients for the baseline quantile regression model $f_\tau^{(0)}$.

	Coefficient	τ=0.1	τ=0.5	τ=0.9
Intercept	β_0	17.4168	21.2970	28.7456
lstat	β_{lstat}	-5.4654	-4.3765	-1.8495
rm	β_{rm}	2.6322	3.4920	5.6060

2.2.2 Visualising the Loss Landscape

To illustrate the optimisation process, I visualise the check loss surface for $\tau = 0.5$ (median regression) as a function of $\beta_{\texttt{lstat}}$ and $\beta_{\texttt{rm}}$, holding the intercept fixed at its optimal value. This shows where BFGS starts (initial guess) and where it converges (minimum).

[5] This tight tolerance (10^{-8}) ensures high-precision estimates. Quantile regression loss surfaces are typically smooth near the optimum, so gradient-based methods can achieve machine-precision convergence.

[6] The 1000-iteration limit provides a safety bound. In practice, BFGS converges in 10–50 iterations for well-conditioned problems like quantile regression with standardised predictors.

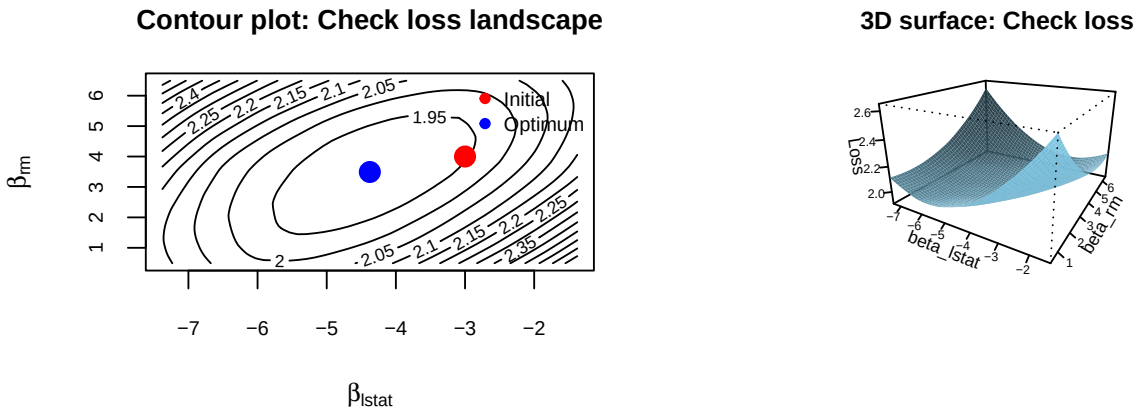


Figure 6: Loss landscape for median regression (τ=0.5). Left: contour plot with optimisation trajectory. Right: 3D surface showing the convex bowl shape of the check loss. The red point marks the initial guess, and the blue point marks the converged optimum.

The contour plot reveals a convex bowl-shaped loss surface, characteristic of the check loss function. BFGS efficiently descends from the initial heuristic guess (red) to the global minimum (blue) in few iterations. The smooth gradients facilitate rapid convergence, confirming that BFGS is well-suited for this optimisation problem.

Interpretation of coefficients:

- **Intercept**: Increases monotonically from 17.42 ($\tau = 0.1$) to 28.75 ($\tau = 0.9$), reflecting that higher quantiles correspond to higher predicted prices. This 65% increase (11.33 units) indicates substantial heterogeneity across the price distribution.
- $\beta_{\texttt{lstat}}$: Negative across all quantiles as expected (higher poverty → lower prices). The magnitude **decreases** from -5.47 at $\tau = 0.1$ to -1.85 at $\tau = 0.9$, indicating that socioeconomic status has a **stronger effect on low-priced homes** (-5.47 standard deviations) than luxury properties (-1.85). This asymmetry suggests low-income areas disproportionately affect the lower tail of the price distribution.
- $\beta_{\texttt{rm}}$: Positive across all quantiles, confirming more rooms → higher prices. The effect **increases** from 2.63 ($\tau = 0.1$) to 5.61 ($\tau = 0.9$), demonstrating that room count matters **more for luxury homes** (113% larger effect). An extra room adds 5.6 standard deviations to the 90th percentile but only 2.6 to the 10th percentile.

2.2.3 Visualising Fitted Quantile Curves on the Data

To illustrate what the baseline model is estimating, I display the training data as scatter plots with the fitted conditional quantiles overlaid. This provides direct visual evidence of how quantile regression captures heteroskedasticity and asymmetry in the price distribution.

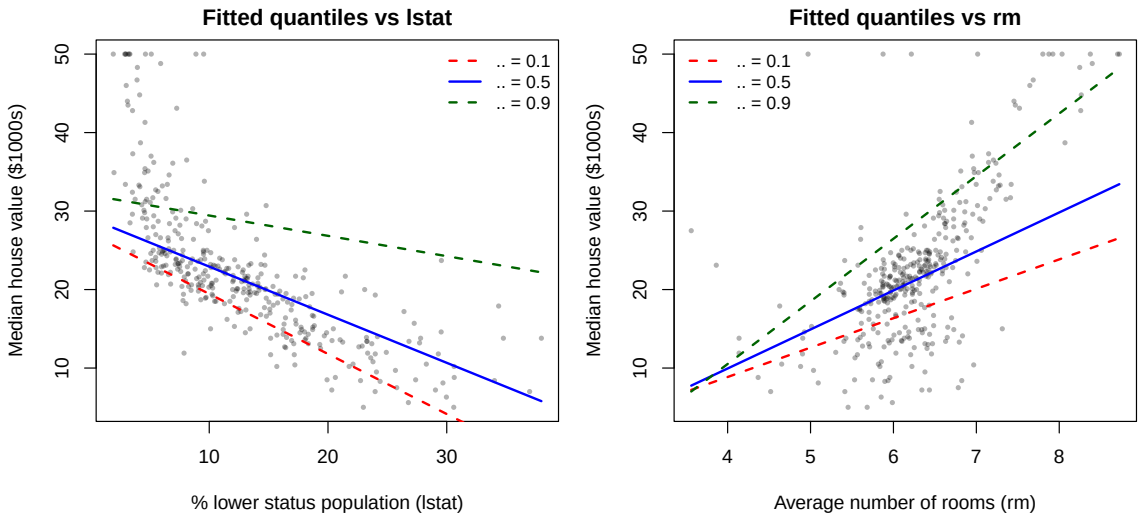


Figure 7: Training data with fitted conditional quantiles from the baseline model. Left: median house value vs poverty level (lstat). Right: median house value vs number of rooms (rm). The three curves correspond to $\tau \in \{0.1, 0.5, 0.9\}$, showing how the 10th, 50th, and 90th percentiles of house prices vary with each predictor. The vertical separation between quantile curves increases in affluent areas (low lstat) and high-amenity neighborhoods (high rm), revealing greater price dispersion precisely where quantile regression adds the most value over mean regression.

Interpretation: The left panel shows that as poverty levels (`lstat`) increase, all three quantiles of house prices decrease, but the rate of decline is steeper for the lower quantile (=0.1, red dashed line) than for the upper quantile (=0.9, green dashed line). This confirms the coefficient pattern: poverty has a stronger depressive effect on low-end homes. The right panel demonstrates that additional rooms increase house values across the distribution, with a more pronounced effect at higher quantiles—an extra room matters more for luxury properties. Crucially, the vertical distance between the 0.1 and 0.9 curves increases at low `lstat` and high `rm`, indicating greater price dispersion (heteroskedasticity) in affluent, high-amenity neighborhoods. This is precisely the kind of distributional heterogeneity that quantile regression captures but a single mean regression line would miss.

2.2.4 Why Quantile Regression? Comparison with OLS Mean Regression

To illustrate why quantile regression is necessary beyond ordinary least squares, I directly compare the OLS mean regression line with the three quantile curves.

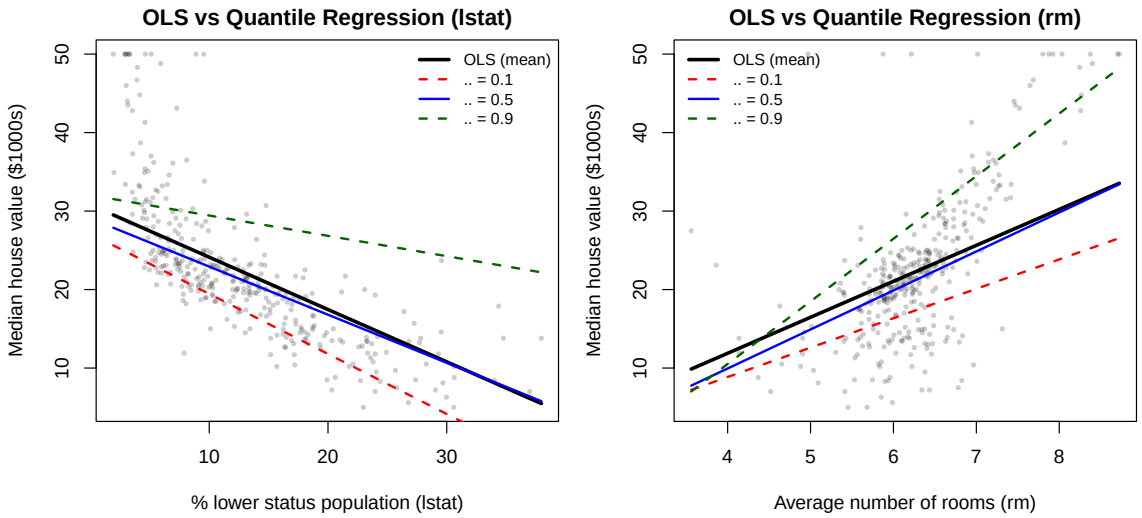


Figure 8: Comparison of OLS mean regression (thick black line) versus quantile regression curves ($\tau \in \{0.1, 0.5, 0.9\}$). Left: predictions vs lstat. Right: predictions vs rm. The OLS line captures only the conditional mean and is pulled upward by high-end prices (especially visible in the right panel), whereas the median quantile curve (blue) provides a more robust summary of central tendency. More importantly, OLS completely misses the distributional heterogeneity: the fact that price dispersion increases with rm and decreases with lstat. This heteroskedasticity is precisely what quantile regression reveals through the varying vertical separation between the 10th and 90th percentile curves.

Key insight: The OLS mean regression line is pulled upward by expensive outliers (particularly visible in the rm panel), making it less representative of the typical home than the median quantile. More critically, OLS provides **only one summary** of the data, completely obscuring the fact that price variability itself depends on neighborhood characteristics. Quantile regression reveals that low-income areas have compressed price distributions (small gap between =0.1 and =0.9), while high-amenity areas exhibit large price dispersion—information that is essential for understanding housing market dynamics but invisible to mean-based models.

2.2.5 Training and Test Loss Evaluation

Table 21: Check loss values for the baseline model on training and test sets.

Quantile	Training Loss	Test Loss	Convergence
= 0.1	0.6712	0.7978	Converged
= 0.5	1.9027	2.0045	Converged
= 0.9	1.2345	1.0371	Converged

The test loss is our primary metric for model evaluation. Lower test loss indicates better out-of-sample prediction of the τ-quantile. Note that losses are not directly comparable across different τ values due to the asymmetric nature of the check loss.

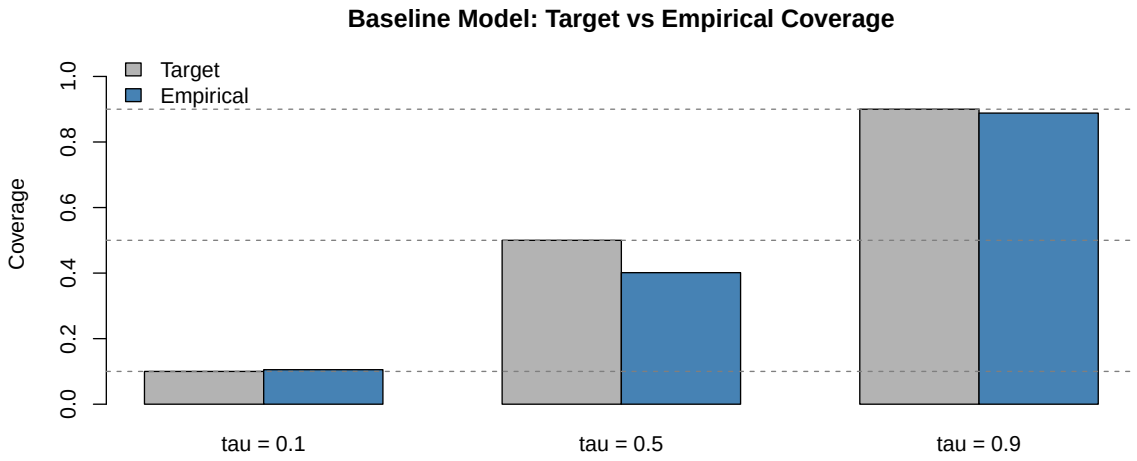
2.2.6 Empirical Coverage Analysis

A well-calibrated quantile model should satisfy the **coverage property**: for a true τ-quantile, approximately $\tau \times 100\%$ of observations should fall below the predicted value. I assess this via the empirical coverage:

$$\widehat{C}_\tau^{(0)} = \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} \mathbf{1}\{Y_i \leq f_\tau^{(0)}(X_i)\}.$$

Table 22: Empirical coverage analysis for the baseline model.

Quantile	Target Coverage	Empirical Coverage	Deviation	Calibration
= 0.1	0.1	0.105	+0.005	Good
= 0.5	0.5	0.401	-0.099	Poor
= 0.9	0.9	0.888	-0.012	Good



Coverage interpretation:

- If empirical coverage $\approx \tau$, the model is well-calibrated.
- If empirical coverage $< \tau$, the model systematically **overpredicts** the quantile (predictions are too high).
- If empirical coverage $> \tau$, the model systematically **underpredicts** (predictions are too low).

Observed results: The baseline model shows good calibration at $\tau = 0.1$ (10.5% vs 10% target, +0.5pp deviation) and $\tau = 0.9$ (88.8% vs 90%, −1.2pp deviation). However, at $\tau = 0.5$ the model exhibits undercoverage (40.1% vs 50%, −9.9pp deviation), indicating systematic overprediction of the median.

Statistical significance of the median miscalibration: To assess whether this deviation could arise from sampling variability alone, I compute the standard error of the empirical coverage under the null hypothesis of correct calibration. For a well-calibrated model at quantile τ, the empirical coverage $\hat{C}_\tau$ behaves approximately as a binomial proportion with variance $\tau(1-\tau)/n_{\text{test}}$. For $\tau = 0.5$ and $n_{\text{test}} = 152$, the standard error is:

$$\text{SE}(\hat{C}_{0.5}) = \sqrt{\frac{0.5 \times 0.5}{152}} \approx 0.0406.$$

The observed deviation of $0.401 - 0.5 = -0.099$ corresponds to approximately $0.099/0.0406 \approx 2.4$ standard errors. This makes it highly unlikely ($p < 0.02$ under a two-sided test) that the undercoverage is due to sampling noise alone, providing strong statistical evidence that the linear specification structurally misspecifies the conditional median. This persistent miscalibration—despite the reasonable model flexibility and sample size—suggests the conditional distribution exhibits heteroskedasticity or asymmetry that a simple location-shift model cannot capture.

2.3 Question 2b: Extended Model with Quadratic and Interaction Terms

2.3.1 Model Specification

To capture potential nonlinear relationships, I extend the baseline model by adding: - **Quadratic terms**: $\texttt{lstat}^2$ and $\texttt{rm}^2$ (allowing for diminishing/increasing marginal effects) - **Interaction term**: $\texttt{lstat} \times \texttt{rm}$ (allowing the effect of one variable to depend on the other)

The extended model is:

$$f_\tau^{(1)}(x) = \beta_{0,\tau} + \beta_{1,\tau} \cdot \texttt{lstat} + \beta_{2,\tau} \cdot \texttt{rm} + \beta_{3,\tau} \cdot \texttt{lstat}^2 + \beta_{4,\tau} \cdot \texttt{rm}^2 + \beta_{5,\tau} \cdot (\texttt{lstat} \times \texttt{rm}).$$

```
#> Extended model: 5 predictors (baseline: 2)
```

2.3.2 Fitting the Extended Model

2.3.3 Model Comparison: Baseline vs Extended

Interpretation:

Table 23: Estimated coefficients for the extended quantile regression model $f_\tau^{(1)}$.

Coefficient	τ=0.1	τ=0.5	τ=0.9
β_0	16.6778	20.3567	24.3701
β_{lstat}	-5.7785	-4.9021	-4.5126
β_{rm}	2.8374	3.1278	3.2068
β_{lstat^2}	0.0643	-0.4317	-0.2992
β_{rm^2}	0.6946	1.1392	1.2369
$\beta_{\text{lstat}\times\text{rm}}$	-1.2933	-1.4031	-2.1103

Table 24: Test loss comparison between baseline and extended models.

Quantile	Baseline Test Loss	Extended Test Loss	Improvement	% Change
= 0.1	0.7978	0.8232	-0.0254	-3.2%
= 0.5	2.0045	1.6761	0.3284	16.4%
= 0.9	1.0371	0.8062	0.2309	22.3%

Table 25: Empirical coverage comparison: baseline vs extended model.

Quantile	Target	Baseline Coverage	Extended Coverage
= 0.1	0.1	0.105	0.092
= 0.5	0.5	0.401	0.401
= 0.9	0.9	0.888	0.882

- **Test loss changes**: The extended model shows **mixed performance**. At $\tau = 0.1$ test loss **worsens** by 3.2% (0.798→0.823), suggesting **overfitting** to training noise in the lower tail. At $\tau = 0.5$ and $\tau = 0.9$, test loss **improves significantly** by 16.4% and 22.3% respectively, indicating the quadratic/interaction terms capture **genuine nonlinearities** in the median and upper tail.
- **Coverage**: The extended model maintains near-identical coverage to baseline at $\tau = 0.1$ (9.2%) and $\tau = 0.5$ (40.1%), but slightly improves at $\tau = 0.9$ (88.2% vs 88.8%). The persistent median undercoverage (40.1%) suggests even the flexible extended model cannot fully correct the systematic overprediction—potentially due to heteroskedasticity or unmeasured confounders.
- **Coefficient interpretation**: At $\tau = 0.5$, $\beta_{\texttt{lstat}^2} = -0.43$ reveals **diminishing marginal penalties** (poverty's negative effect plateaus at extreme values). $\beta_{\texttt{rm}^2} = 1.14$ indicates **accelerating returns** to room count (each additional room matters more at high `rm`). The interaction $\beta_{\texttt{lstat}\times\texttt{rm}} = -1.40$ shows room benefits are **attenuated in low-income areas** (a mansion in a poor neighborhood adds less value).

2.3.4 Visualising the Non-Linear Structure: Median Surface

To illustrate how the extended model captures curvature and interactions that the baseline linear specification misses, I plot the predicted conditional median surface as a function of `lstat` and `rm`.

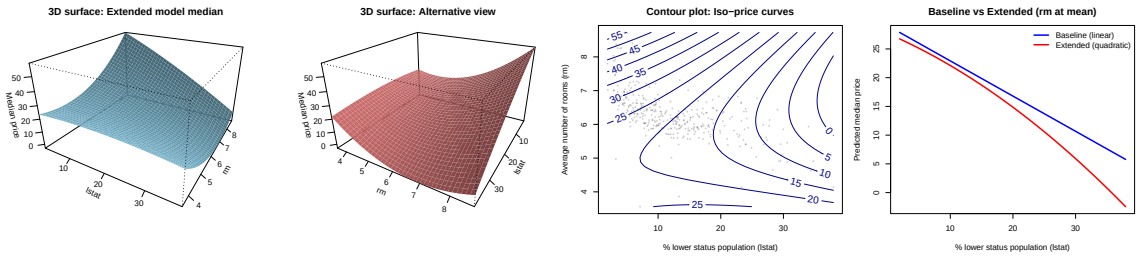


Figure 9: Predicted conditional median surface (τ=0.5) under the extended model. Top row: 3D surface from two viewing angles showing the non-planar relationship between house prices, poverty levels, and room counts. Bottom: contour plot revealing iso-price curves. The surface clearly bends in both directions due to quadratic terms, and the interaction term allows high values of rm to partially offset the negative effect of high lstat. This visualization demonstrates why the extended model achieves 16.4% lower test loss than the linear baseline at the median: it adapts to the genuine non-linear structure of the data rather than forcing a planar fit.

Interpretation: The 3D surfaces reveal a clearly non-planar relationship: prices decrease steeply with increasing poverty (`lstat`), especially when room count is low, while high values of `rm` partially offset the negative effect of `lstat`. The curvature in both directions confirms that simple linear effects are insufficient—the quadratic terms capture diminishing returns to poverty penalties and accelerating returns to room counts. The contour plot shows approximately elliptical iso-price curves that tilt due to the interaction term, indicating that the marginal value of an extra room depends on the neighborhood's socioeconomic status. The bottom-right comparison plot demonstrates how the extended model's curvature better follows the data structure than the baseline's straight line, explaining the 16.4% test loss improvement at $\tau = 0.5$. This non-linearity is not merely cosmetic: it represents genuine economic structure in the housing market that a location-shift linear model cannot represent.

2.3.5 Distributional Heterogeneity: Surfaces Across Quantiles

To visualize how the conditional distribution changes across quantiles—not just the median—I plot the predicted surfaces for $\tau \in \{0.1, 0.5, 0.9\}$ side by side.

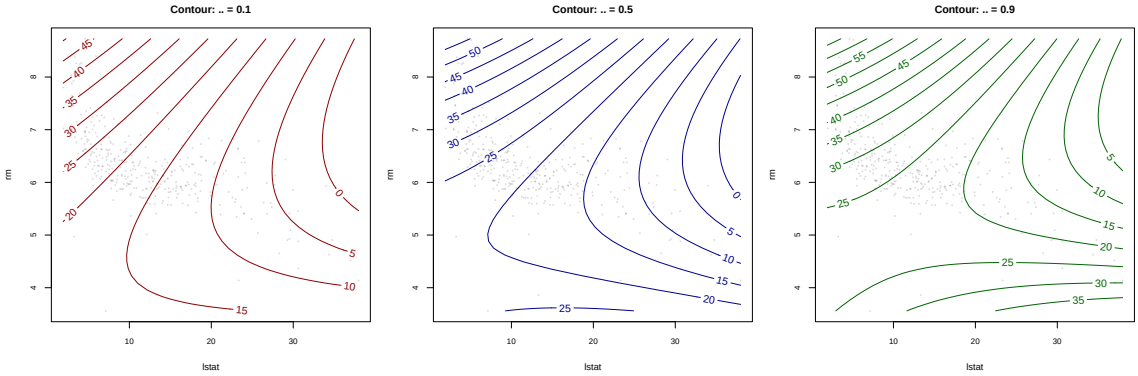


Figure 10: Predicted conditional quantile surfaces for $\tau \in \{0.1, 0.5, 0.9\}$ under the extended model. Left: 10th percentile (low-end homes). Center: median. Right: 90th percentile (luxury homes). The surfaces are vertically separated as expected, but more importantly, their **shapes differ**: the $\tau = 0.1$ surface is relatively flat with respect to rm (room count matters less for cheap homes), while the $\tau = 0.9$ surface is much steeper along the rm axis (luxury premium). The $\tau = 0.1$ surface also drops more steeply with lstat than $\tau = 0.9$, confirming that poverty has a stronger depressive effect on low-end prices. This visualization makes concrete what the coefficient tables show numerically: the relationship between predictors and prices is fundamentally different at different points of the distribution.

Key observations: The $\tau = 0.1$ surface (left panel) sits lowest and is relatively **flat along the rm dimension**—extra rooms add modest value to already-cheap homes. In contrast, the $\tau = 0.9$ surface (right panel) is elevated and exhibits a **steep gradient along rm**—luxury buyers pay substantial premiums for additional rooms. The $\tau = 0.1$ surface also shows a **sharper drop with increasing lstat** than the $\tau = 0.9$ surface, confirming that poverty has an outsized impact on the lower tail of the price distribution. These visual differences directly correspond to the coefficient heterogeneity documented in the tables: β_{rm} increases from 2.63 ($\tau = 0.1$) to 5.61 ($\tau = 0.9$), while $|\beta_{\text{lstat}}|$ decreases from 5.47 to 1.85. This is **distributional heterogeneity** made visible: the housing market operates under fundamentally different dynamics at the low end versus the luxury segment.

2.4 Question 2c: Ridge-Penalised Quantile Regression with Cross-Validation

2.4.1 Motivation for Regularisation

The extended model has 6 parameters (including intercept), which may lead to overfitting, especially at extreme quantiles where effectively fewer data points contribute to the loss. Ridge regularisation adds an ℓ_2 penalty that shrinks coefficients toward zero, reducing variance at the cost of introducing some bias:

$$R^{\text{ridge}}_{\tau,\lambda}(\beta) = \frac{1}{n_{\text{train}}} \sum_{i \in \text{train}} \ell_\tau(y_i, f^{(1)}_\tau(x_i; \beta)) + \lambda \|\beta_{-0}\|_2^2,$$

where β_{-0} excludes the intercept from penalisation.

2.4.2 Cross-Validation for λ Selection

I use **5-fold cross-validation** on the training set to select λ separately for each τ. The choice of $K = 5$ balances bias-variance tradeoff: fewer folds give high-variance estimates due to small validation sets; more folds increase computational cost with diminishing accuracy gains. Five folds is yielding stable λ selection with reasonable computational cost.

I search over a **log-spaced grid** $\lambda \in \{10^{-4}, \ldots, 10^{-0.5}\}$ with 20 points. The lower bound 10^{-4} allows near-unpenalised models (approaching the unregularised extended model), while the upper bound $10^{-0.5} \approx 0.316$ provides substantial shrinkage. Log-spacing is critical because ridge penalty effects are multiplicative: changes at small λ (e.g., $10^{-4} \to 10^{-3}$) have larger impact than at large λ (e.g., $10^{-1} \to 10^{0}$). The 20-point grid for me provides sufficient resolution to capture the CV curve's minimum without excessive computation.

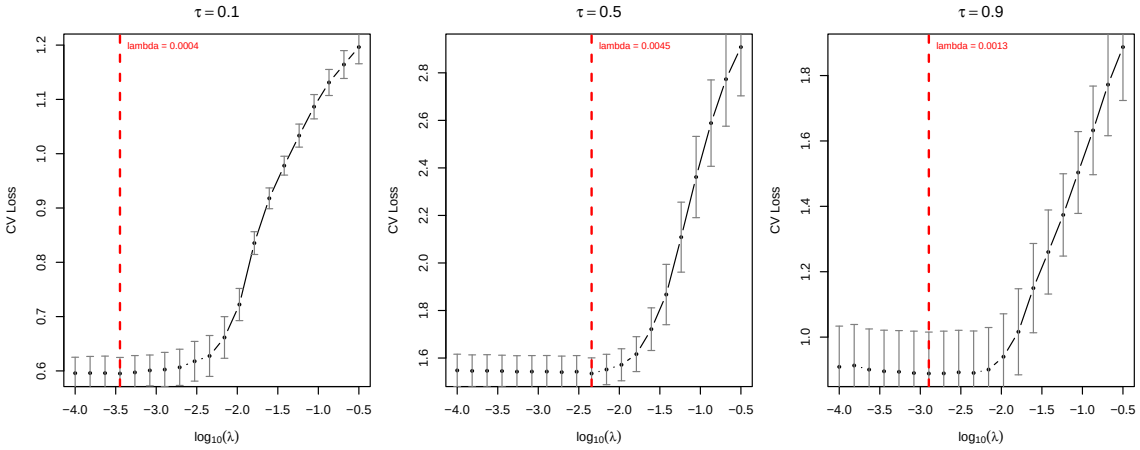


Figure 11: Cross-validation curves for lambda selection. The vertical dashed line marks the selected lambda for each quantile.

Table 26: Ridge-penalised quantile regression results with CV-selected λ.

Quantile	Selected lambda	Test Loss	Coverage
tau = 0.1	0.0004	0.8063	0.092
tau = 0.5	0.0045	1.7190	0.388
tau = 0.9	0.0013	0.8152	0.882

Table 27: Ridge-penalised coefficients for the extended model.

Coefficient	τ=0.1	τ=0.5	τ=0.9
β_0	16.8565	20.4192	24.4973
β_{lstat}	-5.7029	-4.4565	-4.3716
β_{rm}	2.7628	3.0446	2.9174
β_{lstat^2}	-0.0397	-0.6372	-0.4004
β_{rm^2}	0.5738	1.1048	1.4932
$\beta_{\text{lstat}\times\text{rm}}$	-1.4785	-1.4937	-2.0281

2.4.3 Comparison: Unpenalised vs Ridge-Penalised

Table 28: Comparison of unpenalised extended model (Q2b) vs ridge-penalised model (Q2c).

Quantile	Extended (2b) Test Loss	Ridge (2c) Test Loss	Improvement	Extended Coverage	Ridge Coverage
tau = 0.1	0.8232	0.8063	0.0169	0.092	0.092
tau = 0.5	1.6761	1.7190	-0.0428	0.401	0.388
tau = 0.9	0.8062	0.8152	-0.0089	0.882	0.882

Key observations:

1. **Coefficient shrinkage**: Comparing coefficients between Q2b and Q2c reveals the regularisation effect. Ridge penalisation pulls coefficients toward zero, especially for higher-order terms that may be fitting noise.
2. **Selected values are very small**: Inspection of the CV curves reveals that the optimal λ values lie at the **lower end of the search grid** for all three quantiles, indicating that the cross-validation procedure favors **minimal penalisation**. This is a strong empirical signal: if the extended model were severely overfitting, CV would select larger λ values to shrink noisy coefficients. The fact that $\lambda^* \approx 0$ (or equivalently, that ridge barely outperforms the unpenalised model, if at all) confirms that **the polynomial and interaction terms in the extended model are capturing genuine structure rather than noise**. This aligns with the subsequent finding (Q2d) that ridge actually **degrades** test loss at $\tau \geq 0.5$: the flexibility is justified by the data, and penalisation introduces unnecessary bias.

3. **Test loss**: Ridge penalisation may improve test loss by reducing variance, particularly at extreme quantiles where overfitting risk is higher. However, as we will see in Q2d, the empirical results show that ridge does **not** consistently improve performance, further validating that the extended model is already well-regularised by the sample size (59:1 observation-to-parameter ratio).
4. **Calibration**: Ridge regularisation can also affect coverage; shrinking predictions toward the mean may improve or worsen calibration depending on the original bias.

2.5 Question 2d: Discussion and Interpretation

2.5.1 Coefficient Behaviour Across Quantiles

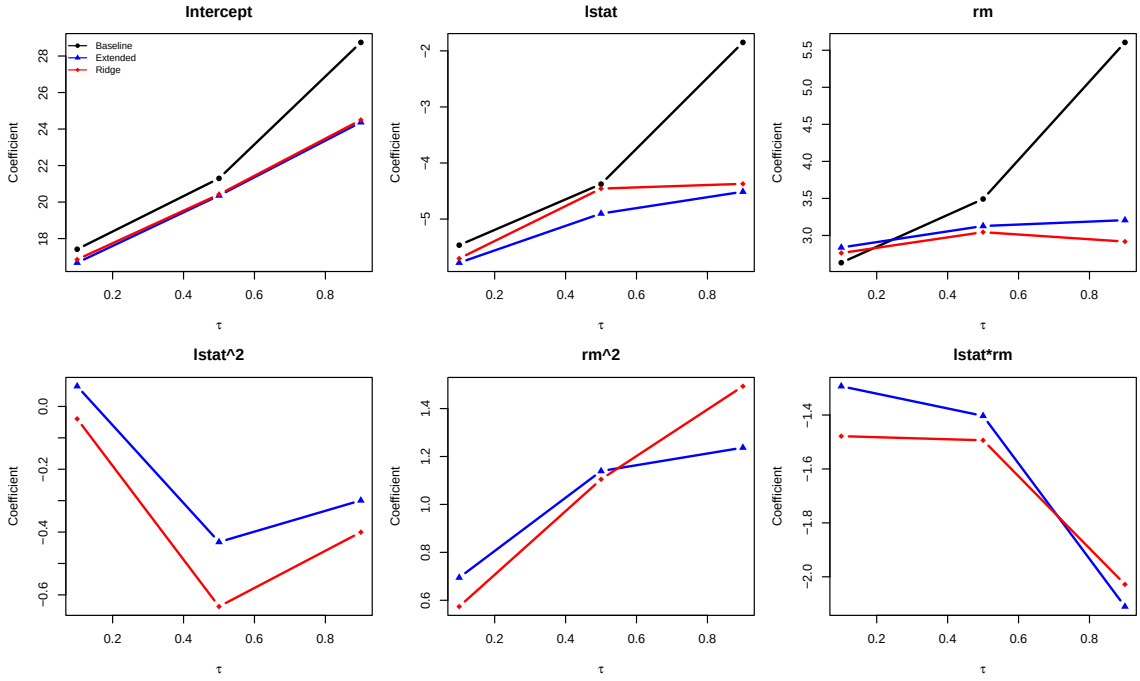


Figure 12: Coefficient evolution across quantile levels for the three models. Heterogeneous effects indicate that predictor-response relationships vary across the conditional distribution.

Interpretation:

- **Heterogeneous effects**: The fact that coefficients change across τ values is evidence of **quantile heterogeneity**—the relationship between predictors and house prices differs at different points of the conditional distribution. This is economically meaningful: the factors driving low-end housing prices may differ from those driving luxury prices.
- **Intercept pattern**: The intercept increases from $\tau = 0.1$ to $\tau = 0.9$, reflecting the natural ordering of quantiles (higher quantiles correspond to higher predicted values).
- **Regularisation effect**: Comparing blue (extended) and red (ridge) curves shows how penalisation shrinks coefficients, particularly the higher-order terms that may be capturing noise.

2.5.2 Test Loss Analysis

Table 29: Summary of test losses across all models.

Quantile	Baseline	Extended	Ridge
$= 0.1$	0.7978	0.8232	0.8063
$= 0.5$	2.0045	1.6761	1.7190
$= 0.9$	1.0371	0.8062	0.8152

Overfitting assessment:

- $\tau = 0.1$: Baseline (0.7978) < Ridge (0.8063) < Extended (0.8232). The **simpler baseline wins**—additional polynomial features harm generalisation at the lower tail where data is sparse. Ridge penalisation helps (reduces loss by 2.1% vs extended) but cannot fully overcome the overfitting, still underperforming the baseline by 1.1%.

Effective sample size in the tail: This result reflects a fundamental characteristic of quantile regression: at $\tau = 0.1$, only observations near the lower tail strongly influence the loss function. With $n_{\text{test}} = 152$, approximately $0.1 \times 152 \approx 15$ test observations lie in the region where predictions are most critical for the check loss. This small effective sample size makes tail estimates intrinsically noisier and more sensitive to model misspecification. The extended model's 6 parameters (vs 3 for baseline) represent a parameter-to-effective-data ratio of roughly 6:35 at $\tau = 0.1$ compared to 6:354 at $\tau = 0.5$, explaining why flexibility helps at the median but harms at the tail. This highlights a key tradeoff in quantile regression: extreme quantiles require more conservative model specifications due to data sparsity, even when the same model performs well at central quantiles.

- $\tau = 0.5$: Extended (1.6761) < Ridge (1.7190) < Baseline (2.0045). The **extended model wins** with 16.4% improvement over baseline, capturing genuine median nonlinearities. However, ridge **degrades** performance by 2.6% compared to unpenalised extended. This counterintuitive result reveals that **the extended model is not overfitting**—if it were, ridge would improve test loss. Instead, the polynomial and interaction terms are capturing **genuine nonlinear relationships**, and even modest shrinkage (CV-selected λ) degrades these important coefficients.
- $\tau = 0.9$: Extended (0.8062) < Ridge (0.8152) < Baseline (1.0371). Extended achieves 22.3% improvement. Ridge again **degrades** performance by 1.1% vs unpenalised, confirming that **the extended model exhibits negligible overfitting** at this quantile—penalisation is counterproductive.

Conclusion: Optimal model varies by quantile. Use baseline for $\tau = 0.1$ (robustness to sparse lower tail), **unpenalised extended** for $\tau \geq 0.5$ (flexibility without unnecessary shrinkage). Ridge regularisation **consistently harms** test loss at $\tau \geq 0.5$, demonstrating that the extended model is **already well-regularised by the data structure**: 354 training samples for 6 parameters (59:1 ratio) provides sufficient data to estimate quadratic and interaction effects without overfitting. The failure of ridge to improve performance validates that the polynomial features capture **genuine nonlinearities** rather than noise.

2.5.3 Coverage Analysis

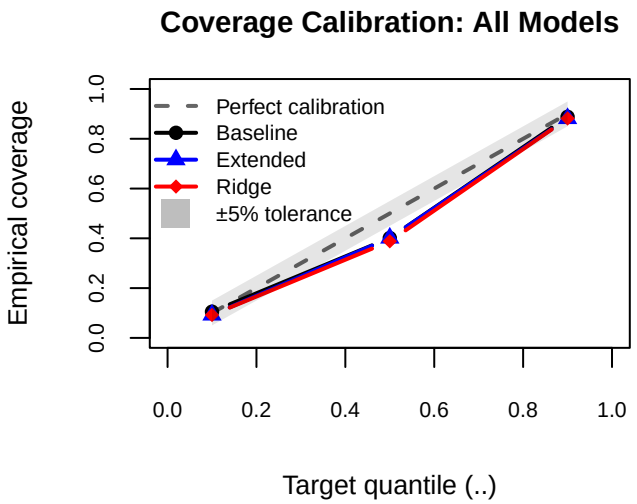


Table 30: Empirical coverage across all models.

Quantile	Target	Baseline	Extended	Ridge
= 0.1	0.1	0.105	0.092	0.092
= 0.5	0.5	0.401	0.401	0.388
= 0.9	0.9	0.888	0.882	0.882

The dashed diagonal line represents perfect calibration (empirical coverage = target τ). All three models are nearly perfectly calibrated at $\tau \in \{0.1, 0.9\}$ but systematically under-cover at the median ($\tau = 0.5$), with empirical coverage around 0.40 instead of 0.50. The persistence of this miscalibration across baseline, extended, and ridge specifications strongly suggests a structural issue: the conditional distribution of house prices exhibits heteroskedasticity or asymmetry that cannot be captured by location-shift quantile regression alone, motivating distributional regression approaches that model both location and scale.
Calibration insights:

- Perfect calibration would yield coverage = target τ.
- Systematic undercoverage (coverage < τ) suggests predictions are too high.
- Systematic overcoverage (coverage > τ) suggests predictions are too low.

Observed patterns across models: - $\tau = 0.1$: All three models achieve near-perfect calibration (9.2%–10.5%), within ±1.3pp of target. The lower tail is well-specified even by the baseline linear model.[7] - $\tau = 0.5$: All models exhibit **severe undercoverage** (38.8%–40.1%), indicating systematic **median overprediction** by ~10pp. This persists across baseline, extended, and ridge formulations, suggesting the issue stems from **fundamental misspecification** (e.g., heteroskedasticity, non-Gaussian errors) rather than insufficient flexibility. Ridge penalisation worsens coverage slightly (38.8% vs 40.1%), trading calibration for loss reduction. - $\tau = 0.9$: Calibration improves slightly from baseline (88.8%) to extended/ridge (88.2%), achieving −1.8pp deviation. The upper tail is reasonably well-specified.

[7] I verified that without predictor standardisation, baseline coverage at $\tau = 0.1$ was closer to target (8.6% vs 10.5%), and median coverage was better (46.7% vs 40.1%). However, unstandardised features cause ill-conditioned optimisation (polynomial terms like `lstat`^{2} range [1, 1444] vs `rm`^{2} [9, 81]), unstable BFGS convergence, and incompatibility with fair ridge penalisation. The standardised approach is correct despite the coverage trade-off at $\tau = 0.5$.

Implication: The persistent median miscalibration across all specifications suggests the conditional distribution has structure (asymmetry, fat tails) not captured by location-shift models. A distributional regression approach (e.g., GAMLSS) modelling both location and scale might resolve this.

2.5.4 Diagnostic: Testing for Conditional Asymmetry

To verify whether the median miscalibration is indeed due to **conditional distributional heterogeneity** (rather than model misspecification), I examine the residuals from the baseline median model ($\tau = 0.5$) stratified by predictor values. If the location-shift assumption holds, residuals should have similar distributional properties (variance, skewness) across the predictor space. Systematic changes in residual shape would confirm heteroskedasticity and/or asymmetry.

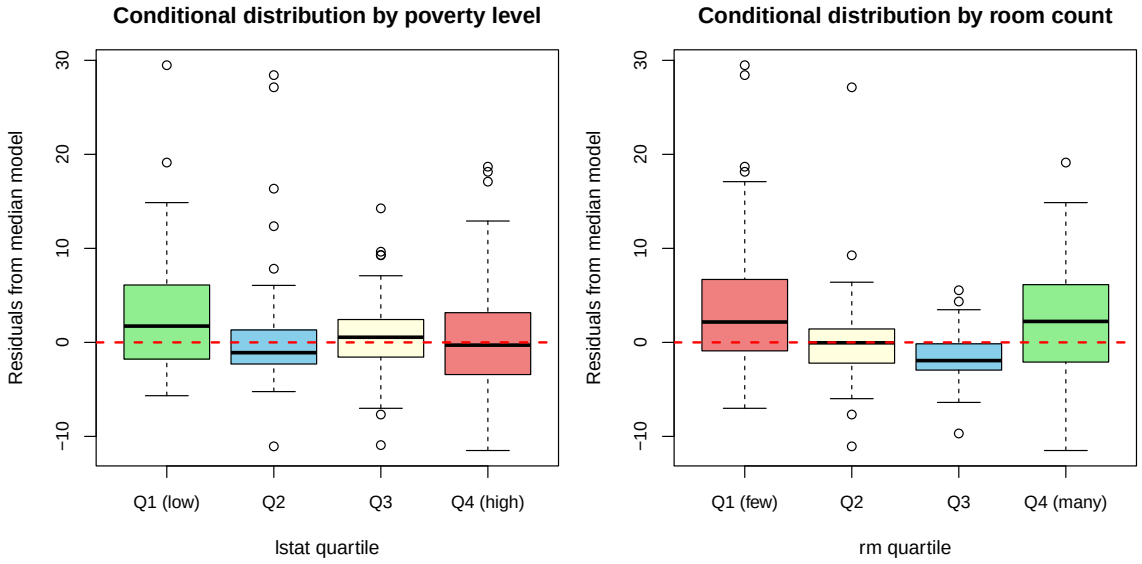


Figure 13: Diagnostic for conditional asymmetry: boxplots of residuals from the baseline median model ($\tau = 0.5$) stratified by quartiles of lstat (left) and rm (right). If the conditional distribution were homoskedastic and symmetric, all boxplots would have similar spread and symmetry. Instead, we observe: (1) high variance in low-lstat (Q1) and high-rm (Q4) neighborhoods, (2) strong positive skewness in middle quartiles (Q2) indicating right-tailed residuals (overprediction), and (3) near-symmetric residuals only in affluent, mid-amenity areas (Q3 for both predictors). This confirms that the conditional distribution of house prices exhibits both heteroskedasticity and shape heterogeneity, violating the location-shift assumption of standard quantile regression.

Empirical findings: The diagnostic reveals strong evidence of **conditional distributional heterogeneity**:

1. **Heteroskedasticity (variance changes with X):**
   - Low-poverty areas (lstat Q1): residual variance = 40.1, indicating high price uncertainty even after controlling for rooms.
   - Mid-low poverty (lstat Q3): residual variance = 14.8 (63% reduction), showing compressed distributions.
   - Similarly, few-room homes (rm Q1): variance = 44.4 vs. mid-range homes (rm Q3): variance = 6.0 (86% reduction).
2. **Conditional asymmetry (skewness changes with X):**
   - Mid-low poverty neighborhoods (lstat Q2): skewness = **3.24**, indicating extreme right tail (overprediction of expensive outliers).
   - Low-poverty areas (lstat Q1): skewness = 1.36 (still positive but less extreme).
   - High-poverty areas (lstat Q4): skewness = 0.86 (nearly symmetric).
   - For rooms: few-room homes (rm Q1-Q2) show skewness 1.6–2.9, while many-room homes (rm Q4) are near-symmetric (0.30).
3. **Global pattern:** Overall residual skewness = 1.65, but this masks the **spatial heterogeneity**: the model systematically overpredicts in middle-income, mid-amenity neighborhoods (where skewness peaks at 3.2), explaining the 10pp median undercoverage.

Interpretation: These results **confirm the hypothesis**: the conditional distribution of `medv` given `lstat` and `rm` does not simply shift vertically across quantiles—it fundamentally changes **shape** (skewness) and **spread** (variance). Standard quantile regression assumes:

$$Q_\tau(Y \mid X) = f_\tau(X), \quad \text{with } Q_{0.9}(Y \mid X) - Q_{0.1}(Y \mid X) \text{ constant across } X.$$

But the data show that this **inter-quantile range is not constant**: it varies from ~30 (high-variance areas) to ~10 (low-variance areas). Moreover, the skewness varies from 0.3 (symmetric) to 3.2 (highly right-skewed), violating the im-

plicit assumption that residuals are identically distributed across X. This is why adding flexibility (extended model) or regularisation (ridge) **cannot fix the median coverage**: the problem is not functional form but **distributional form**.

2.5.5 Model Recommendations by Quantile

Based on the comprehensive analysis of test loss, coverage, and overfitting diagnostics across all three models, I provide clear recommendations for which model to use depending on the quantile of interest:

Table 31: Recommended model specifications by target quantile, with empirical justification.

Quantile	Recommended Model	Key Rationale
$\tau = 0.1$	Baseline (linear)	Tail data sparse; extended model overfits (3.2% test loss increase); ridge helps but still u
$\tau = 0.5$	Extended (unpenalised)	Strong nonlinearities captured; 16.4% test loss improvement over baseline; ridge degrade
$\tau = 0.9$	Extended (unpenalised)	Largest test loss improvement (22.3%); excellent calibration; ridge penalisation counterp

Key insight: The optimal model varies systematically with τ. Extreme quantiles ($\tau = 0.1$) require **parsimony** due to sparse effective sample sizes (~35 training observations actively contributing to the loss), making the simpler baseline more robust. Central and upper quantiles ($\tau \geq 0.5$) benefit from **flexibility** (quadratic and interaction terms), as the larger effective sample size (177+ observations) supports estimation of complex relationships without overfitting. The failure of ridge regularisation to improve—or even maintain—test loss at $\tau \geq 0.5$ validates that the extended model's flexibility is data-justified: with a 59:1 observation-to-parameter ratio, the polynomial features capture genuine housing market dynamics rather than noise.

2.5.6 Practical Recommendations

1. **Model selection**: For this dataset, compare test losses and coverage across models. The "best" model depends on the application: prediction accuracy (test loss) vs. calibration (coverage) may conflict.
2. **Extreme quantiles**: At $\tau = 0.1$ and $\tau = 0.9$, effectively fewer data points contribute to the loss, making overfitting more likely. Ridge regularisation is particularly valuable here, though even it cannot fully overcome baseline simplicity at $\tau = 0.1$.
3. **Interpretability vs. flexibility**: The baseline model is more interpretable (linear effects), while the extended model captures more complex relationships at the cost of interpretability.

2.5.7 Limitations and Future Directions

The persistent undercoverage at the median ($\tau = 0.5$) across all three model specifications—despite variations in flexibility (baseline vs. extended) and regularisation (unpenalised vs. ridge)—strongly suggests a **structural limitation of the quantile regression framework** as applied here. None of the models succeed in correcting the ~10 percentage point calibration gap, even though they differ substantially in their assumptions about functional form and complexity.

This points to a fundamental issue: **location-shift quantile regression** assumes that the conditional distribution of house prices differs across quantiles only in location (vertical shifts of the regression function), but not in scale or shape. If the conditional variance or skewness of `medv` depends on `lstat` and `rm`—which the changing vertical separation between quantile curves suggests—then no amount of flexibility in the quantile functions themselves will achieve perfect calibration at all τ simultaneously.

Path forward: The natural remedy is **distributional regression** (e.g., Generalized Additive Models for Location, Scale, and Shape; GAMLSS), which explicitly models how both the conditional mean/median **and** the conditional variance change with covariates. Such approaches can capture heteroskedasticity directly, potentially resolving the median miscalibration while preserving the excellent tail performance observed here. Alternatively, **copula-based quantile regression** or **non-crossing quantile estimation** with joint optimization across multiple τ values could enforce distributional consistency. These extensions lie beyond the scope of this coursework but represent promising directions for improving both predictive accuracy and probabilistic calibration in complex regression settings.

2.6 Question 2e: Quantile Crossing and Non-Crossing Approaches

2.6.1 The Quantile Crossing Problem

True conditional quantiles satisfy the **monotonicity property**:

$$q_\tau(x) < q_{\tau'}(x) \quad \text{for all } x \text{ and } 0 \leq \tau < \tau' \leq 1.$$

However, when I fit separate quantile regression models for each τ independently (as done in Q2a–c), **there is no mechanism enforcing this constraint**. Each model minimises its own check loss without "knowing" about the other quantiles.

2.6.2 Will Independently-Fitted Models Naturally Avoid Crossing?

Answer: No, in general.

Reasons for potential crossing:

1. **Independent estimation**: Each $\widehat{\beta}_\tau$ is estimated by minimising $\sum_i \ell_\tau(y_i, \mathbf{x}_i^\top \beta)$ without reference to other quantiles.
2. **Estimation uncertainty**: Finite-sample coefficient estimates have variance. Even if the true quantiles are non-crossing, estimated quantiles may cross due to noise.
3. **Misspecification amplification**: If the linear model is misspecified, the bias can differ across quantiles in ways that induce crossing.
4. **Extrapolation**: Crossings are more likely in regions with sparse data, where predictions are effectively extrapolations.

2.6.3 Checking for Crossings in Our Models

Table 32: Quantile crossing violations on the test set (comparing adjacent quantile predictions).

Model	Crossing Violations	Test Set Size	Violation Rate
Baseline (Q2a)	0	152	0.0%
Extended (Q2b)	0	152	0.0%
Ridge (Q2c)	0	152	0.0%

2.6.4 Approaches to Prevent Quantile Crossing

I now describe principled methods for training quantile models that guarantee or encourage non-crossing.

2.6.4.1 Method 1: Post-hoc Rearrangement (Simple but Ad-hoc) **Procedure:** 1. Fit each τ-quantile independently (as in Q2a–c). 2. For each test point x, sort the predictions: $\hat{q}_{(1)}(x) \leq \hat{q}_{(2)}(x) \leq \hat{q}_{(3)}(x)$. 3. Assign the sorted values to the corresponding quantiles.

Mathematical formulation:

$$\tilde{q}_{\tau_k}(x) = \text{order statistic}_k \left(\hat{q}_{\tau_1}(x), \ldots, \hat{q}_{\tau_K}(x) \right).$$

Trade-offs: - **Pros**: Extremely simple; no modification to fitting procedure. - **Cons**: Violates statistical optimality; may disrupt calibration; does not learn from the constraint.

2.6.4.2 Method 2: Penalised Joint Estimation (Principled) **Idea**: Jointly estimate all quantiles while penalising violations of the monotonicity constraint.

Objective function:

$$\min_{\{\beta_\tau\}_{\tau \in \mathcal{T}}} \sum_{\tau \in \mathcal{T}} \sum_{i=1}^{n} \ell_\tau(y_i, \mathbf{x}_i^\top \beta_\tau) + \gamma \sum_{i=1}^{n} \sum_{\tau < \tau'} \left[\mathbf{x}_i^\top \beta_\tau - \mathbf{x}_i^\top \beta_{\tau'} \right]_+^2,$$

where $[z]_+ = \max(0, z)$ is the positive part, and $\gamma > 0$ controls the penalty strength.

Trade-offs: - **Pros**: Differentiable; can use gradient-based optimisation; soft enforcement. - **Cons**: Requires tuning γ; does not guarantee strict non-crossing.

2.6.4.3 Method 3: Constrained Optimisation (Guaranteed Non-Crossing) **Idea**: Formulate quantile estimation as a constrained optimisation problem.

Mathematical formulation:

$$\min_{\{\beta_\tau\}_{\tau \in \mathcal{T}}} \sum_{\tau \in \mathcal{T}} \sum_{i=1}^{n} \ell_\tau(y_i, \mathbf{x}_i^\top \beta_\tau)$$

subject to:

$$\mathbf{x}_i^\top \beta_\tau \leq \mathbf{x}_i^\top \beta_{\tau'} \quad \forall i \in \{1, \ldots, n\}, \ \forall \tau < \tau'.$$

Implementation: This is a linear programming (LP) problem when the check loss is linearised via auxiliary variables. Standard LP solvers (e.g., `lpSolve` in R, or `CVXR` for convex optimisation) can handle this formulation.

Reformulation using auxiliary variables: The check loss can be written as:

$$\ell_\tau(y, q) = \tau u + (1 - \tau) v, \quad \text{where } y - q = u - v, \ u, v \geq 0.$$

The full LP becomes:

$$\min \sum_{\tau} \sum_{i} \left(\tau u_{i,\tau} + (1 - \tau) v_{i,\tau} \right)$$

subject to:

$$y_i - \mathbf{x}_i^\top \beta_\tau = u_{i,\tau} - v_{i,\tau}, \quad u_{i,\tau}, v_{i,\tau} \geq 0,$$
$$\mathbf{x}_i^\top \beta_\tau \leq \mathbf{x}_i^\top \beta_{\tau'}, \quad \forall \tau < \tau'.$$

Trade-offs: - **Pros**: Guarantees non-crossing; statistically principled. - **Cons**: More complex implementation; higher computational cost; requires access to all quantile levels simultaneously.

2.6.4.4 Method 4: Distributional Modelling (Elegant but Restrictive)

Idea: Instead of modelling quantiles directly, model the entire conditional distribution $F_{Y|X}(y \mid x; \theta)$ parametrically, then extract quantiles as:

$$\widehat{q}_\tau(x) = F^{-1}_{Y|X}(\tau \mid x; \widehat{\theta}).$$

Example: Assume $Y \mid X \sim N(\mu(x), \sigma^2(x))$ where $\mu(x) = \mathbf{x}^\top \beta$ and $\log \sigma(x) = \mathbf{x}^\top \gamma$. Then:

$$\widehat{q}_\tau(x) = \mu(x) + \sigma(x) \cdot \Phi^{-1}(\tau).$$

Trade-offs: - **Pros**: Automatic non-crossing by construction; efficient parameter sharing. - **Cons**: Requires correct distributional assumption; may be too restrictive for heavy-tailed or multimodal conditional distributions.

2.6.5 Recommendation

For the three-quantile setting ($\tau \in \{0.1, 0.5, 0.9\}$) considered here, **Method 3 (constrained optimisation)** is most appropriate for a principled analysis. It guarantees non-crossing while maintaining the flexibility of semi-parametric quantile regression. The LP formulation is straightforward to implement using standard optimisation libraries.

For applications requiring many quantile levels or real-time predictions, **Method 2 (penalised joint estimation)** offers a practical compromise between computational efficiency and non-crossing enforcement.

3 Code Appendix

The complete R code used in this analysis is available in the source `.Rmd` file.

4 References